计算几何

Morning Flower and Evening Oath

XCPC vector 模板分册

建设稿：验证状态与适用边界见各条目及仓库 verification/。

目录

使用约定 iii

第一章 二维计算几何基础 1
1.1 浮点点向量、圆与交点结果 . . . 1
1.2 有限二进制浮点极角精确排序 . . . 3
1.3 点在直线上的投影 . . . 5
1.4 浮点两直线交点 . . . 6
1.5 浮点直线与圆交点 . . . 8
1.6 浮点两圆交点 . . . 10
1.7 两圆交面积 . . . 13
1.8 整数点向量与线段判定 . . . 14
1.9 整数输入直线与圆交点 . . . 16
1.10 整数输入两圆交点 . . . 18
1.11 整数多边形有向面积 . . . 19
1.12 点在整数简单多边形内 . . . 22
1.13 圆与简单多边形相交面积 . . . 23
1.14 两圆公切线与过点切线 . . . 25
1.15 整数圆公切线与精确切点排序 . . . 27
1.16 一般多边形面积并：整数精确判定 . . . 30
1.17 圆面积并：未覆盖圆弧积分 . . . 36
1.18 圆覆盖重数面积：恰好覆盖 k 次 . . . 40

第二章 三维计算几何基础 44
2.1 整数三维共线、共面与方向 . . . 44
2.2 三维凸包：精确方向与退化处理 . . . 46
2.3 整数凸多面体：体积重心与表面距离 . . . 56
2.4 实数三维向量与夹角 . . . 61
2.5 三维直线投影、线段距离与绕轴旋转 . . . 64
2.6 三维平面投影与分类求交 . . . 67

第三章 距离 71
3.1 点到闭线段的距离 . . . 71

第四章 Pick 定理 73
4.1 Pick 定理：格点、剖分与孔洞 . . . 73

第五章 凸包 75
5.1 凸多边形外点的两条切线 . . . 75
5.2 整数凸包：单调链 . . . 79
5.3 点在整数凸包内（二分） . . . 82
5.4 凸多边形 Minkowski 和 . . . 83
5.5 支撑点查询恢复下凸包 . . . 85

第六章 扫描线 **89**
6.1 整数轴对齐矩形面积并 . . . 89

第七章 旋转卡壳 **91**
7.1 整数凸包直径：旋转卡壳 . . . 91

第八章 半平面交 **94**
8.1 整数半平面的有界多边形交 . . . 94

第九章 平面最近点对 **98**
9.1 整数最近点对（精确平方距离） . . . 98
9.2 浮点平面最近点对 . . . 101

第十章 随机增量法 **103**
10.1 随机增量最小覆盖圆 . . . 103

第十一章 配套知识与验证范围 **105**
11.1 数值算法与几何 . . . 105

第十二章 计算几何：精确判定与适用范围 **106**
12.1 定点凸包、直径和极角排序 . . . 106
12.2 整数系数的有界半平面交 . . . 107
12.3 最近点对、Minkowski 和与三维整数判定 . . . 108
12.4 浮点最近点对 . . . 109
12.5 公切线与过点切线 . . . 110
12.6 随机增量最小覆盖圆 . . . 111
12.7 圆与简单多边形相交面积 . . . 112
12.8 最近点对编号与多边形闭集查询 . . . 112
12.9 整数圆交点：精确判定与浮点构造 . . . 113
12.10 公切线：切点次序也需要精确判定 . . . 113
12.11 圆交点：容差分类与坐标误差 . . . 114
12.12 三维投影、旋转与平面求交的数值约定 . . . 115
12.12.1 原三维片段中需要避开的错误 . . . 116

使用约定

C++20，默认包含 bits/stdc++.h 与 cassert，并使用 std 命名空间。下标、区间及数值范围以各条目说明为准。DFS 保持递归，需满足题目栈空间条件。

本册主目录沿用 OI Wiki 官方导航的大、中、小层级，各页下列本库实现。PBDS 与 GNU rope 按队内约定纳入平衡树。组合条目与相关挂靠仍见仓库分类审计。跨类引用的已收录组件附在依赖部分；源头文件中的依赖仍须一并检查。

本册尚非全量完成稿；传统版评测仅为历史档案，当前只维护 vector 代码。

第一章　二维计算几何基础

1.1　浮点点向量、圆与交点结果

R 为 long double；Point 支持向量加减、数乘和数除（除数非零）。dot/cross/norm/perp 为基本运算，norm 使用 hypotl；Circle 保存圆心和非负半径。Kind 为 none、one、two、infinite、degenerate；Result 同时保存类别与有限交点。输入须有限，尺度须使后续平方、乘积不溢出或下溢；排序不使用 epsilon。具体容差与退化约定见各函数。

```
struct RealPlane
{
    using R = long double;

    struct Point
    {
        R x = 0, y = 0;

        Point operator+(Point b) const { return {x + b.x, y + b.y}; }

        Point operator-(Point b) const { return {x - b.x, y - b.y}; }

        Point operator*(R k) const { return {x * k, y * k}; }

        Point operator/(R k) const { return {x / k, y / k}; }
    };

    struct Circle
    {
        Point o;
        R r;
    };
```

```
    enum class Kind
    {
        none,
        one,
        two,
        infinite,
        degenerate
    };

    struct Result
    {
        Kind kind;
        vector<Point> p;
    };

    static R dot(Point a, Point b) { return a.x * b.x + a.y * b.y; }

    static R cross(Point a, Point b) { return a.x * b.y - a.y * b.x; }

    static R norm(Point a) { return hypotl(a.x, a.y); }

    static Point perp(Point a) { return {-a.y, a.x}; }
};
```

模板题使用：AOJ CGL_2_A

题意：每组给两条非退化直线，平行输出 2，垂直输出 1，否则输出 0。输入为绝对值不超过 10000 的整数，点积与叉积在此范围可由浮点精确表示，所以直接比较 0；这不适用于一般实数或更大整数。只演示点向量与 dot/cross，不涵盖圆及 Result 分类。

所需模板：RealPlane（第 1 页，1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://onlinejudge.u-aizu.ac.jp/problems/CGL_2_A

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int q;
    cin >> q;
    while (q--)
    {
        RealPlane::Point a, b, c, d;
        cin >> a.x >> a.y >> b.x >> b.y >> c.x >> c.y >> d.x >> d.y;
        auto u = b - a, v = d - c;
        if (RealPlane::cross(u, v) == 0)
            cout << 2 << '\n';
        else if (RealPlane::dot(u, v) == 0)
            cout << 1 << '\n';
        else
            cout << 0 << '\n';
    }
}
```

1.2 有限二进制浮点极角精确排序

代码依赖：RealPlane（第 1 页，1.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

RealPolarLess{}(a,b) 沿用 RealPlane::Point 存储，输入为实际已存入的有限二进制 long double，尾数 ≤64 位且 uint128 可用；禁 fast-math 及 FTZ/DAZ。原点最前，正 x 轴起逆时针 $[0,2\pi)$，同向按长度递增，同点及 ±0 等价，满足严格弱序。用 frexp/ldexp 拆尾数并以 uint128 精确比较乘积，不做浮点叉乘、不用 epsilon。每次 O(1)，无分配。十进制输入先舍入；不修复调用前的坐标差，不替代其他浮点几何谓词。product/cross_sign 是有限输入下的内部辅助。

```
struct RealPolarLess
{
    using R = RealPlane::R;
    using P = RealPlane::Point;
    using U = __uint128_t;
    static_assert(numeric_limits<R>::radix == 2 && numeric_limits<R>::digits <= 64);

    struct Product
    {
        int sign;
        long long exponent;
        U value;
    };

    static Product product(R a, R b)
    {
        if (a == 0 || b == 0) return {0, 0, 0};
        int x, y;
        constexpr int p = numeric_limits<R>::digits;
        uint64_t u = ldexp(frexp(abs(a), &x), p);
        uint64_t v = ldexp(frexp(abs(b), &y), p);
        U m = U(u) * v;
        uint64_t hi = m >> 64;
        int bits = hi ? 64 + bit_width(hi) : bit_width(uint64_t(m));
        return {signbit(a) == signbit(b) ? 1 : -1,
                (long long)x + y - 2 * p + bits, m << (128 - bits)};
    }

    static int cross_sign(P a, P b)
    {
        Product u = product(a.x, b.y), v = product(a.y, b.x);
        if (u.sign != v.sign) return u.sign < v.sign ? -1 : 1;
        if (!u.sign) return 0;
        int c = u.exponent != v.exponent ? (u.exponent < v.exponent ? -1 : 1)
              : u.value != v.value ? (u.value < v.value ? -1 : 1) : 0;
        return u.sign * c;
    }
```

接上页同一结构体：极角比较接口。

```
    bool operator()(P a, P b) const
    {
        assert(isfinite(a.x) && isfinite(a.y) && isfinite(b.x) && isfinite(b.y));
        bool az = a.x == 0 && a.y == 0, bz = b.x == 0 && b.y == 0;
        if (az || bz) return az && !bz;
        auto half = [](P p) { return p.y < 0 || (p.y == 0 && p.x < 0); };
        if (half(a) != half(b)) return half(a) < half(b);
        int c = cross_sign(a, b);
        if (c) return c > 0;
        return a.x != 0 ? abs(a.x) < abs(b.x) : abs(a.y) < abs(b.y);
    }
};
```

接口用法演示：自定义有限浮点向量极角排序

演示约定：自定义协议，非 OJ 题：0≤n≤200000，接着读 n 对坐标，须成功转换为有限 long double；读入失败、n 越界或非有限值返回 1，不作额外词法或 EOF 校验。输出 1-based 原编号：原点最前，正 x 轴起逆时针 $[0,2\pi)$，同向按长度，完全相同的数值再按原编号。±0 视为相同。比较的是转换后的二进制值，不是舍入前十进制实数；沿用本节尾数 ≤64 位及正常浮点环境条件。std::sort 共 O(n log(n+1)) 次比较，向量存储 O(n)。

所需模板：RealPlane（第 1 页，1.1 节）、RealPolarLess（第 3 页，1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://github.com/gsh20040816/CPCtemplate/blob/main/src/compact/real_polar.hpp

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    if (!(cin >> n) || n < 0 || n > 200000) return 1;
    using P = RealPlane::Point;
    vector<pair<P, int>> points(n);
    for (int i = 0; i < n; i++)
    {
        auto &p = points[i].first;
        if (!(cin >> p.x >> p.y) || !isfinite(p.x) || !isfinite(p.y)) return 1;
        points[i].second = i + 1;
    }
    RealPolarLess less;
    sort(points.begin(), points.end(), [&](const auto &a, const auto &b)
    {
        return less(a.first, b.first) ||
               (!less(b.first, a.first) && a.second < b.second);
    });
    for (const auto &p : points) cout << p.second << ' ';
    cout << '\n';
}
```

1.3 点在直线上的投影

代码依赖：RealPlane（第 1 页，1.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

line_projection(p,a,b) 返回 p 在过 a、b 直线上的正交投影；若 a=b，则约定返回 a。类型为 RealPlane::Point。输入有限且尺度适当，平方和与点积须能可靠表示；不使用 epsilon 把短线段自动视为零。O(1) 时间和空间。

```
RealPlane::Point
line_projection(RealPlane::Point p, RealPlane::Point a, RealPlane::Point b)
{
    using G = RealPlane;
    using R = G::R;
    using Point = G::Point;
    Point v = b - a;
    R d = G::dot(v, v);
    if (d == 0) return a;
    return a + v * (G::dot(p - a, v) / d);
}
```

模板题使用：AOJ CGL_1_A

题意：固定直线两端点，对每个点输出其在无限直线上的投影。题目保证两端点不同，整数坐标绝对值不超过 10000，q 不超过 1000；投影可在线段外。接口参数为 (p,a,b)，不截断到线段；a=b 时模板返回 a 属于额外边界。

所需模板：RealPlane（第 1 页，1.1 节）、line_projection（第 5 页，1.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://onlinejudge.u-aizu.ac.jp/problems/CGL_1_A

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    RealPlane::Point a, b;
    int q;
    cin >> a.x >> a.y >> b.x >> b.y >> q;
    cout << fixed << setprecision(12);
    while (q--)
    {
        RealPlane::Point p;
        cin >> p.x >> p.y;
        auto h = line_projection(p, a, b);
        cout << h.x << ' ' << h.y << '\n';
    }
}
```

1.4 浮点两直线交点

代码依赖：RealPlane（第 1 页，1.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

line_intersection_real(a,b,c,d,eps=1e-12L) 返回 RealPlane::Result；eps 必须为正。端点重合使任一直线退化时返回 degenerate。按方向叉积与方向长度乘积的相对阈值判断平行，再按位置阈值区分 none 与 infinite；唯一交点返回 one。输入有限且尺度适当；近似分类不等同于精确共线判定。O(1)。

```
RealPlane::Result line_intersection_real(RealPlane::Point a,
                                         RealPlane::Point b,
                                         RealPlane::Point c,
                                         RealPlane::Point d,
                                         long double eps = 1e-12L)
{
    using G = RealPlane;
    using R = G::R;
    using Point = G::Point;
    using Kind = G::Kind;
    assert(eps > 0);
    Point u = b - a, v = d - c;
    R nu = G::norm(u), nv = G::norm(v);
    if (nu == 0 || nv == 0) return {Kind::degenerate, {}};
    R det = G::cross(u, v);
    if (fabsl(det) <= eps * nu * nv)
        return {fabsl(G::cross(c - a, u)) <= eps * nu * max(R(1), G::norm(c - a))
                    ? Kind::infinite
                    : Kind::none,
                {}};
    return {Kind::one, {a + u * (G::cross(c - a, v) / det)}};
}
```

模板题使用：AOJ CGL_2_C

题意：输出两条线段的唯一交点；题目保证非平行且相交，q 不超过 1000，整数坐标绝对值不超过 10000，因此可直接取 one 结果。模板计算支撑直线交点，不自动检查交点在线段内；一般输入须先检查 kind。整数方向叉积非零时绝对值至少 1，大于本题范围下默认相对阈值。

所需模板：RealPlane（第 1 页，1.1 节）、line_intersection_real（第 6 页，1.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://onlinejudge.u-aizu.ac.jp/problems/CGL_2_C

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int q;
    cin >> q;
    cout << fixed << setprecision(12);
    while (q--)
    {
        RealPlane::Point a, b, c, d;
        cin >> a.x >> a.y >> b.x >> b.y >> c.x >> c.y >> d.x >> d.y;
        auto ans = line_intersection_real(a, b, c, d);
        assert(ans.kind == RealPlane::Kind::one);
        cout << ans.p[0].x << ' ' << ans.p[0].y << '\n';
    }
}
```

1.5 浮点直线与圆交点

代码依赖：RealPlane（第 1 页，1.1 节）、line_projection（第 5 页，1.3 节）。抄写时一并准备；标准库类型不列入算法依赖。

line_circle_intersections(a,b,c,eps=1e-12L) 返回 Result；c 为 RealPlane::Circle，半径非负，eps 非负；以下相对阈值描述适用于 eps>0。a=b 返回 degenerate；按投影点到圆心的距离和半径的相对阈值区分 none、one、two。点圆允许；只有正半径同心圆才涉及无限交点，本函数不会返回 infinite。依赖 line_projection，O(1)。输入有限且尺度适当，接近相切时按容差归类。默认容差不能直接满足 AOJ CGL_7_D 的所有合法输入；近切时归并交点可能超出坐标误差限。eps=0 为显式无容差模式：直接计算平方判别式，不主动合并近切点；一般实数仍可能发生舍入误判、溢出或下溢。整数坐标绝对值 ≤10000、整数半径 0..10000，二进制 long double 尾数 ≥64 位且正常最近舍入、禁用 fast-math 时，中间整数精确。交点坐标仍为浮点近似；其他输入不继承这一保证。

```
RealPlane::Result line_circle_intersections(RealPlane::Point a,
                                            RealPlane::Point b,
                                            RealPlane::Circle c,
                                            long double eps = 1e-12L)
{
    using G = RealPlane;
    using R = G::R;
    using Point = G::Point;
    using Kind = G::Kind;
    assert(eps >= 0);
    assert(c.r >= 0);
    Point v = b - a;
    if (eps == 0)
    {
        R q = G::dot(v, v);
        if (q == 0) return {Kind::degenerate, {}};
        R z = G::cross(v, a - c.o);
        R d = c.r * c.r * q - z * z;
        if (d < 0) return {Kind::none, {}};
        Point h = c.o + G::perp(v) * (z / q);
        if (d == 0) return {Kind::one, {h}};
        Point w = v * (sqrtl(d) / q);
        return {Kind::two, {h - w, h + w}};
    }
    R len = G::norm(v);
    if (len == 0) return {Kind::degenerate, {}};
    Point h = line_projection(c.o, a, b);
    R d = G::norm(h - c.o), tol = eps * max(d, c.r);
    if (d > c.r + tol) return {Kind::none, {}};
    if (fabsl(d - c.r) <= tol) return {Kind::one, {h}};
    R t = sqrtl(max(R(0), (c.r - d) * (c.r + d)));
    Point w = v * (t / len);
    return {Kind::two, {h - w, h + w}};
}
```

模板题使用：AOJ CGL_7_D —浮点接口无容差模式

题意：整数坐标绝对值 ≤10000、整数半径 1..10000，保证交点存在；每条直线两端点不同，1≤q≤1000。直接调用浮点接口并显式传 eps=0，不合并近切交点；仅在二进制 long double 尾数至少 64 位、正常最近舍入且禁用 fast-math 的平台采用此用法。此整数域的平方判别式精确，坐标仍为浮点构造；一般实数不继承精确分类保证。相切重复同一点，按普通 (x,y) 字典序输出，不能用 epsilon 排序；默认 eps>0 接口保持原近似语义。

所需模板：RealPlane（第 1 页，1.1 节）、line_projection（第 5 页，1.3 节）、line_circle_intersections（第 8 页，1.5 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.u-aizu.ac.jp/onlinejudge/description.jsp?id=CGL_7_D

```
int main()
{
    static_assert(numeric_limits<long double>::radix == 2 &&
                  numeric_limits<long double>::digits >= 64);
#ifdef __FAST_MATH__
#error This example requires normal floating arithmetic
#endif
    if (fegetround() != FE_TONEAREST) return 1;
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    RealPlane::Circle c;
    int q;
    cin >> c.o.x >> c.o.y >> c.r >> q;
    cout << fixed << setprecision(12);
    while (q--)
    {
        RealPlane::Point a, b;
        cin >> a.x >> a.y >> b.x >> b.y;
        auto ans = line_circle_intersections(a, b, c, 0);
        assert(ans.p.size() == 1 || ans.p.size() == 2);
        if (ans.p.size() == 1) ans.p.push_back(ans.p[0]);
        if (tie(ans.p[1].x, ans.p[1].y) < tie(ans.p[0].x, ans.p[0].y))
            swap(ans.p[0], ans.p[1]);
        cout << ans.p[0].x << ' ' << ans.p[0].y << ' ' << ans.p[1].x << ' '
             << ans.p[1].y << '\n';
    }
}
```

1.6 浮点两圆交点

代码依赖：RealPlane（第 1 页，1.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

circle_intersections(a,b,eps=1e-12L) 返回 Result；半径非负，eps 非负；以下相对阈值描述适用于 eps>0。两个完全相同的正半径圆返回 infinite；同位置两个点圆返回 one；同心不同半径返回 none。其余用距离和半径的相对阈值分类，点圆单独处理，接近相切可返回 one。输入有限且尺度适当，O(1) 时间空间；返回的有限交点仍受浮点构造误差影响。默认容差不能直接满足 AOJ CGL_7_E 的所有合法输入；近切时归并交点可能超出坐标误差限。eps=0 为显式无容差模式：直接计算平方判别式，不主动合并近切点；一般实数仍可能发生舍入误判、溢出或下溢。整数坐标绝对值 ≤10000、整数半径 0..10000，二进制 long double 尾数 ≥64 位且正常最近舍入、禁用 fast-math 时，中间整数精确。交点坐标仍为浮点近似；其他输入不继承这一保证。

```
RealPlane::Result
circle_intersections(RealPlane::Circle a, RealPlane::Circle b, long double eps = 1e-12L)
{
    using G = RealPlane;
    using R = G::R;
    using Point = G::Point;
    using Kind = G::Kind;
    assert(eps >= 0);
    assert(a.r >= 0 && b.r >= 0);
    Point v = b.o - a.o;
    if (eps == 0)
    {
        R q = G::dot(v, v);
        if (q == 0)
        {
            if (a.r != b.r) return {Kind::none, {}};
            if (a.r == 0) return {Kind::one, {a.o}};
            return {Kind::infinite, {}};
        }
        R k = q + a.r * a.r - b.r * b.r;
        R d = 4 * a.r * a.r * q - k * k;
        if (d < 0) return {Kind::none, {}};
        Point h = a.o + v * (k / (2 * q));
        if (d == 0) return {Kind::one, {h}};
        Point w = G::perp(v) * (sqrtl(d) / (2 * q));
        return {Kind::two, {h - w, h + w}};
    }
```

默认正容差分支（接上页同一函数）：

```
    R d = G::norm(v);
    R tol = eps * max({d, a.r, b.r});
    // Infinite intersections require identical positive-radius circles.
    if (d == 0)
    {
        if (a.r != b.r) return {Kind::none, {}};
        if (a.r == 0) return {Kind::one, {a.o}};
        return {Kind::infinite, {}};
    }
    if (a.r == 0 && b.r == 0) return {Kind::none, {}};
    if (a.r == 0 || b.r == 0)
    {
        R radius = max(a.r, b.r);
        if (fabsl(d - radius) > tol) return {Kind::none, {}};
        return {Kind::one, {a.r == 0 ? a.o : b.o}};
    }
    if (d > a.r + b.r + tol || d < fabsl(a.r - b.r) - tol) return {Kind::none, {}};
    R x = (d * d + (a.r - b.r) * (a.r + b.r)) / (2 * d);
    Point h = a.o + v * (x / d);
    // Near-concentric circles must not be mistaken for internal tangency.
    if (fabsl(x) > a.r + tol) return {Kind::none, {}};
    if (fabsl(fabsl(x) - a.r) <= tol) return {Kind::one, {h}};
    R y = sqrtl(max(R(0), (a.r - x) * (a.r + x)));
    Point w = G::perp(v) * (y / d);
    return {Kind::two, {h - w, h + w}};
}
```

模板题使用：AOJ CGL_7_E —浮点接口无容差模式

题意：整数坐标绝对值 ≤10000、整数半径 1..10000，保证交点存在；两圆圆心不同。直接调用浮点接口并显式传 eps=0，不合并近切交点；仅在二进制 long double 尾数至少 64 位、正常最近舍入且禁用 fast-math 的平台采用此用法。此整数域的平方判别式精确，坐标仍为浮点构造；一般实数不继承精确分类保证。相切重复同一点，按普通 (x,y) 字典序输出，不能用 epsilon 排序；默认 eps>0 接口保持原近似语义。

所需模板：RealPlane（第 1 页，1.1 节）、circle_intersections（第 10 页，1.6 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.u-aizu.ac.jp/onlinejudge/description.jsp?id=CGL_7_E

```
int main()
{
    static_assert(numeric_limits<long double>::radix == 2 &&
                  numeric_limits<long double>::digits >= 64);
#ifdef __FAST_MATH__
#error This example requires normal floating arithmetic
#endif
    if (fegetround() != FE_TONEAREST) return 1;
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    RealPlane::Circle a, b;
    cin >> a.o.x >> a.o.y >> a.r >> b.o.x >> b.o.y >> b.r;
    auto ans = circle_intersections(a, b, 0);
    assert(ans.p.size() == 1 || ans.p.size() == 2);
    if (ans.p.size() == 1) ans.p.push_back(ans.p[0]);
    if (tie(ans.p[1].x, ans.p[1].y) < tie(ans.p[0].x, ans.p[0].y))
        swap(ans.p[0], ans.p[1]);
    cout << fixed << setprecision(12) << ans.p[0].x << ' ' << ans.p[0].y << ' '
         << ans.p[1].x << ' ' << ans.p[1].y << '\n';
}
```

1.7 两圆交面积

代码依赖：RealPlane（第 1 页，1.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

circle_overlap_area(a,b) 返回 long double 面积，半径非负。分离或外切返回 0；内含时返回小圆面积，含点圆和同心圆。部分相交计算两个圆弓形面积，小角度使用展开式减轻相消；acos 入参截断至 [-1,1]。不使用交点分类容差；要求有限且尺度适当的输入。O(1) 时间空间，不把浮点输出视为精确面积。

```
RealPlane::R circle_overlap_area(RealPlane::Circle a, RealPlane::Circle b)
{
    using G = RealPlane;
    using R = G::R;
    assert(a.r >= 0 && b.r >= 0);
    R d = G::norm(a.o - b.o), pi = acosl(-1.L);
    if (d >= a.r + b.r) return 0;
    if (d <= fabsl(a.r - b.r)) return pi * min(a.r, b.r) * min(a.r, b.r);
    R difference = (a.r - b.r) * (a.r + b.r);
    R x = acosl(clamp((d * d + difference) / (2 * d * a.r), R(-1), R(1)));
    R y = acosl(clamp((d * d - difference) / (2 * d * b.r), R(-1), R(1)));
    auto segment = [](R angle)
    {
        if (angle < 1e-3L)
        {
            R z = angle * angle;
            return angle * z * (R(2) / 3 + z * (-R(2) / 15 + z * R(4) / 315));
        }
        return angle - sinl(2 * angle) / 2;
    };
    return a.r * a.r * segment(x) + b.r * b.r * segment(y);
}
```

模板题使用：AOJ CGL_7_I

题意：两个圆的公共面积，整数坐标绝对值 $\leq 10^4$、半径 $1..10^4$。覆盖相离、内含、同心与部分相交；输出面积的绝对误差须小于 1e-6。

所需模板：RealPlane（第 1 页，1.1 节）、circle_overlap_area（第 13 页，1.7 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.u-aizu.ac.jp/onlinejudge/description.jsp?id=CGL_7_I

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    RealPlane::Circle a, b;
    cin >> a.o.x >> a.o.y >> a.r >> b.o.x >> b.o.y >> b.r;
    cout << fixed << setprecision(12) << circle_overlap_area(a, b) << '\n';
}
```

1.8 整数点向量与线段判定

坐标绝对值不超过 10^{12}，使用 long long 存点、__int128_t 计算叉积、点积与距离平方。Point 按 x、y 字典序比较；cross(a,b,c) 为 ab 与 ac 的叉积，dot 为点积。on_segment 包含端点及退化线段；intersect 判断闭线段有交点。PolarLess 从正 x 轴按 $[0,2\pi)$ 逆时针极角排序，同向按长度平方递增；零向量排在最前。同点等价，比较器满足严格弱序。按其他起点排序应整体循环移动，LC 的 $(-\pi,\pi]$ 顺序仅将 y<0 后缀移到前面。全部单次运算 O(1)，不使用 epsilon。

```
struct IntegerPlane
{
    using ll = long long;
    using I = __int128_t;

    // Contract: |coordinate|<=1e12. Differences/products widened before arithmetic.
    struct Point
    {
        ll x = 0, y = 0;

        bool operator<(Point b) const { return tie(x, y) < tie(b.x, b.y); }

        bool operator==(Point b) const { return x == b.x && y == b.y; }
    };

    static I cross(Point a, Point b, Point c)
    {
        return (I(b.x) - a.x) * (I(c.y) - a.y) - (I(b.y) - a.y) * (I(c.x) - a.x);
    }

    static I dot(Point a, Point b, Point c)
    {
        return (I(b.x) - a.x) * (I(c.x) - a.x) + (I(b.y) - a.y) * (I(c.y) - a.y);
    }

    static I dist2(Point a, Point b) { return dot(a, b, b); }

    static int sign(I x) { return (x > 0) - (x < 0); }

    static bool on_segment(Point p, Point a, Point b)
    {
        return cross(a, b, p) == 0 && dot(p, a, b) <= 0;
    }

    static bool intersect(Point a, Point b, Point c, Point d)
    {
        int x = sign(cross(a, b, c)), y = sign(cross(a, b, d));
        int z = sign(cross(c, d, a)), w = sign(cross(c, d, b));
        if (!x && on_segment(c, a, b)) return true;
        if (!y && on_segment(d, a, b)) return true;
        if (!z && on_segment(a, c, d)) return true;
        if (!w && on_segment(b, c, d)) return true;
        return x * y < 0 && z * w < 0;
    }
```

极角比较器（接上页同一结构体）：

```
    struct PolarLess
    {
        static int half(Point p) { return p.y < 0 || (p.y == 0 && p.x < 0); }

        bool operator()(Point a, Point b) const
        {
            if (half(a) != half(b)) return half(a) < half(b);
            I c = cross({0, 0}, a, b);
            if (c) return c > 0;
            return dist2({0, 0}, a) < dist2({0, 0}, b);
        }
    };
};
```

模板题使用：Library Checker sort_points_by_argument

题意：将最多 20 万个整数点按 $(-\pi,\pi]$ 的极角排序，坐标绝对值 $\le 10^9$，约定原点角度为 0，同角顺序任意。PolarLess 从正 x 轴按 $[0,2\pi)$ 排序，再把 y<0 的后缀整体移到前面；负 x 轴留在最后，不要把它和负 y 半平面一起前移。仅调用整数点与极角比较器。

所需模板：IntegerPlane（第 14 页，1.8 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/sort_points_by_argument

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<IntegerPlane::Point> p(n);
    for (auto &v : p) cin >> v.x >> v.y;
    sort(p.begin(), p.end(), IntegerPlane::PolarLess{});
    auto first = find_if(p.begin(), p.end(), [](auto v) { return v.y < 0; });
    rotate(p.begin(), first, p.end());
    for (auto v : p) cout << v.x << ' ' << v.y << '\n';
}
```

1.9 整数输入直线与圆交点

代码依赖：IntegerPlane（第 14 页，1.8 节）、RealPlane（第 1 页，1.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

line_circle_i64(a,b,o,r)：a、b 定直线，o 为圆心。依赖 IntegerPlane::Point 及 RealPlane::Result。所有输入坐标绝对值 $\le 10^9$，半径在 $0..10^9$；先用 int128 精确判定交点数，再用 long double 构造坐标。a=b 返回 degenerate；相离 none、相切 one、相交 two。结果 p 不排序，one 只含一个点，none/infinite/degenerate 为空。零半径按点圆处理。O(1) 时间和空间。精确的是分类，坐标仍有舍入误差；在 10^9 尺度不承诺绝对 1e-6。AOJ 整数范围 10^4 内的驱动按字典序输出，相切点重复一次。本地高精度与消毒器测试不等于线上 AC。

```
RealPlane::Result line_circle_i64(IntegerPlane::Point a,
                                  IntegerPlane::Point b,
                                  IntegerPlane::Point o,
                                  long long r)
{
    using I = __int128_t;
    using R = RealPlane::R;
    using K = RealPlane::Kind;
    for (auto p : {a, b, o})
        assert(-1000000000 <= p.x && p.x <= 1000000000 && -1000000000 <= p.y &&
               p.y <= 1000000000);
    assert(0 <= r && r <= 1000000000);
    I x = I(b.x) - a.x, y = I(b.y) - a.y;
    I q = x * x + y * y;
    if (q == 0) return {K::degenerate, {}};
    I c = x * (I(a.y) - o.y) - y * (I(a.x) - o.x);
    I d = I(r) * r * q - c * c;
    if (d < 0) return {K::none, {}};
    RealPlane::Point v{R(x), R(y)}, center{R(o.x), R(o.y)};
    auto h = center + RealPlane::perp(v) * (R(c) / R(q));
    if (d == 0) return {K::one, {h}};
    auto w = v * (sqrtl(R(d)) / R(q));
    return {K::two, {h - w, h + w}};
}
```

模板题使用：AOJ CGL_7_D

题意：整数圆心/半径及直线，坐标绝对值 $\le 10^4$、半径 $1..10^4$，q≤1000。题目保证直线非退化且至少一个交点；精确分类后构造坐标，按 x/y 字典序输出，相切点复制一次。绝对误差要求小于 1e-6。

所需模板：IntegerPlane（第 14 页，1.8 节）、RealPlane（第 1 页，1.1 节）、line_circle_i64（第 16 页，1.9 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.u-aizu.ac.jp/onlinejudge/description.jsp?id=CGL_7_D

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    IntegerPlane::Point o;
    long long r;
    int q;
    cin >> o.x >> o.y >> r >> q;
    cout << fixed << setprecision(12);
    while (q--)
    {
        IntegerPlane::Point a, b;
        cin >> a.x >> a.y >> b.x >> b.y;
        auto ans = line_circle_i64(a, b, o, r);
        assert(ans.p.size() == 1 || ans.p.size() == 2);
        if (ans.p.size() == 1) ans.p.push_back(ans.p[0]);
        if (tie(ans.p[1].x, ans.p[1].y) < tie(ans.p[0].x, ans.p[0].y))
            swap(ans.p[0], ans.p[1]);
        cout << ans.p[0].x << ' ' << ans.p[0].y << ' ' << ans.p[1].x << ' '
             << ans.p[1].y << '\n';
    }
}
```

1.10 整数输入两圆交点

代码依赖：IntegerPlane（第 14 页，1.8 节）、RealPlane（第 1 页，1.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

circle_intersections_i64(a,r,b,s)：a、b 为圆心。依赖 IntegerPlane::Point 及 RealPlane::Result。所有输入坐标绝对值 $\le 10^9$，半径在 $0..10^9$；先用 int128 精确判定交点数，再用 long double 构造坐标。相离或严格内含返回 none、相切 one、相交 two；同心等正半径返回 infinite，同心零半径返回 one。结果 p 不排序，one 只含一个点，none/infinite/degenerate 为空。零半径按点圆处理。O(1) 时间和空间。精确的是分类，坐标仍有舍入误差；在 10^9 尺度不承诺绝对 1e-6。AOJ 整数范围 10^4 内的驱动按字典序输出，相切点重复一次。本地高精度与消毒器测试不等于线上 AC。

```
RealPlane::Result circle_intersections_i64(IntegerPlane::Point a,
                                           long long r,
                                           IntegerPlane::Point b,
                                           long long s)
{
    using I = __int128_t;
    using R = RealPlane::R;
    using K = RealPlane::Kind;
    for (auto p : {a, b})
        assert(-1000000000 <= p.x && p.x <= 1000000000 && -1000000000 <= p.y &&
               p.y <= 1000000000);
    assert(0 <= r && r <= 1000000000 && 0 <= s && s <= 1000000000);
    I x = I(b.x) - a.x, y = I(b.y) - a.y;
    I q = x * x + y * y;
    RealPlane::Point center{R(a.x), R(a.y)}, v{R(x), R(y)};
    if (q == 0)
    {
        if (r != s) return {K::none, {}};
        if (r == 0) return {K::one, {center}};
        return {K::infinite, {}};
    }
    I k = q + I(r) * r - I(s) * s;
    I d = 4 * I(r) * r * q - k * k;
    if (d < 0) return {K::none, {}};
    auto h = center + v * (R(k) / (2 * R(q)));
    if (d == 0) return {K::one, {h}};
    auto w = RealPlane::perp(v) * (sqrtl(R(d)) / (2 * R(q)));
    return {K::two, {h - w, h + w}};
}
```

模板题使用：AOJ CGL_7_E

题意：整数圆心/半径，坐标绝对值 $\le 10^4$、半径 $1..10^4$；不同圆心且至少一个交点。按 x/y 字典序输出，相切点复制一次；不使用 eps 排序。绝对误差要求小于 1e-6。

所需模板：IntegerPlane（第 14 页，1.8 节）、RealPlane（第 1 页，1.1 节）、circle_intersections_i64（第 18 页，1.10 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.u-aizu.ac.jp/onlinejudge/description.jsp?id=CGL_7_E

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    IntegerPlane::Point a, b;
    long long r, s;
    cin >> a.x >> a.y >> r >> b.x >> b.y >> s;
    auto ans = circle_intersections_i64(a, r, b, s);
    assert(ans.p.size() == 1 || ans.p.size() == 2);
    if (ans.p.size() == 1) ans.p.push_back(ans.p[0]);
    if (tie(ans.p[1].x, ans.p[1].y) < tie(ans.p[0].x, ans.p[0].y))
        swap(ans.p[0], ans.p[1]);
    cout << fixed << setprecision(12) << ans.p[0].x << ' ' << ans.p[0].y << ' '
         << ans.p[1].x << ' ' << ans.p[1].y << '\n';
}
```

1.11 整数多边形有向面积

代码依赖：IntegerPlane（第 14 页，1.8 节）。抄写时一并准备；标准库类型不列入算法依赖。

polygon_area2(p) 返回 ___int128_t 的有向面积两倍；逆时针为正，反向为负。输入按边界顺序排列，隐式闭合；空集或不足三个点返回 0。单个多边形点数不超过 10^5、坐标绝对值不超过 10^{12}，确保求和在有符号 128 位内。时间 O(n)，额外空间 O(1)。

```
IntegerPlane::I polygon_area2(const vector<IntegerPlane::Point> &p)
{
    using G = IntegerPlane;
    using I = G::I;
    I ans = 0;
    int n = (int)p.size();
    for (int i = 0; i < n; i++)
        ans += I(p[i].x) * p[(i + 1) % n].y - I(p[i].y) * p[(i + 1) % n].x;
    return ans;
}
```

模板题使用：AOJ CGL_3_A

题意：给出逆时针简单多边形，3 至 100 点，整数坐标绝对值不超过 10000，输出面积并保留一位小数。无需凸包，凹多边形也适用。接口返回有向二倍面积，逆时针为正；取绝对值后按奇偶精确打印.0 或.5。本题范围允许将半面积转为 long long；一般 int128 结果不可任意缩窄。

所需模板：IntegerPlane（第 14 页，1.8 节）、polygon_area2（第 19 页，1.11 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://onlinejudge.u-aizu.ac.jp/problems/CGL_3_A

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<IntegerPlane::Point> p(n);
    for (auto &a : p) cin >> a.x >> a.y;
    auto s = polygon_area2(p);
    if (s < 0) s = -s;
    cout << (long long)(s / 2) << (s % 2 ? ".5\n" : ".0\n");
}
```

应用题补充：CSES 2193 Polygon Lattice Points

题意：简单非退化整数多边形，无孔；3≤n≤100000，坐标绝对值 ≤10^9，按边界顺序给点，方向不限。输出严格内部与边界格点数。已有有向二倍面积取绝对值，边界用逐边 gcd，Pick 分子和输出用 128 位；输入已保证简单性，此程序不验证自交。仅组合应用，不新增面积算法覆盖。

所需模板：IntegerPlane（第 14 页，1.8 节）、polygon_area2（第 19 页，1.11 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://cses.fi/problemset/task/2193

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<IntegerPlane::Point> p(n);
    for (auto &a : p) cin >> a.x >> a.y;
    using I = IntegerPlane::I;
    I s = polygon_area2(p), b = 0;
    if (s < 0) s = -s;
    for (int j = 0; j < n; ++j)
    {
        auto a = p[j], c = p[(j + 1) % n];
        b += gcd(abs(a.x - c.x), abs(a.y - c.y));
    }
    auto print = [](I x)
    {
        string s;
        do
        {
            s += char('0' + x % 10);
            x /= 10;
        } while (x);
        reverse(s.begin(), s.end());
        cout << s;
    };
    print((s - b + 2) / 2);
    cout << ' ';
    print(b);
    cout << '\n';
}
```

1.12 点在整数简单多边形内

代码依赖：IntegerPlane（第 14 页，1.8 节）。抄写时一并准备；标准库类型不列入算法依赖。

polygon_contains(p,q) 返回 0 外部、1 边界、2 内部，输入简单多边形，可顺时针或逆时针，不重复首点。空多边形返回 0；单点与线段按闭集判断边界。绕数算法先判线段，再按半开纵坐标区间累计，避免顶点重复计数。坐标范围同 IntegerPlane，O(n) 时间、O(1) 空间。

```
int polygon_contains(const vector<IntegerPlane::Point> &p, IntegerPlane::Point q)
{
    using G = IntegerPlane;
    using Point = G::Point;
    using I = G::I;
    int winding = 0, n = (int)p.size();
    for (int i = 0; i < n; i++)
    {
        Point a = p[i], b = p[(i + 1) % n];
        if (G::on_segment(q, a, b)) return 1;
        I c = G::cross(a, b, q);
        if (a.y <= q.y && b.y > q.y && c > 0) ++winding;
        if (a.y > q.y && b.y <= q.y && c < 0) --winding;
    }
    return winding ? 2 : 0;
}
```

模板题使用：AOJ CGL_3_C

题意：给定简单多边形和查询点，逐个输出 0 外部、1 边界、2 内部；允许凹多边形，按题目给出的逆时针边界顺序读入，不要求重建凸包。n 至 100、q 至 1000，坐标绝对值至 10000；核心也接受反向边界。

所需模板：IntegerPlane（第 14 页，1.8 节）、polygon_contains（第 22 页，1.12 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://onlinejudge.u-aizu.ac.jp/problems/CGL_3_C

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n;
    vector<IntegerPlane::Point> p(n);
    for (auto &v : p) cin >> v.x >> v.y;
    cin >> q;
    while (q--)
    {
        IntegerPlane::Point v;
        cin >> v.x >> v.y;
        cout << polygon_contains(p, v) << '\n';
    }
}
```

1.13 圆与简单多边形相交面积

代码依赖：RealPlane（第 1 页，1.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

area(points,circle) 返回非负相交面积；多边形须简单，允许凹多边形、顺时针/逆时针和相邻重复顶点。少于三点或半径零返回零。edge(a,b,r) 返回圆心在原点时有向边 a 到 b 的面积贡献，r 必须非负。按交点分段，内部用三角形，外部用有向扇形；补偿求和，O(n) 时间、O(1) 额外空间。依赖浮点几何的 Point/Circle 和向量运算。输入须有限、尺度适当，平移到圆心后作差、乘积和面积均须可表示；极小交叠及接近相切时仍有浮点误差。

```
struct CirclePolygon
{
    using G = RealPlane;
    using R = G::R;
    using Point = G::Point;
    using Circle = G::Circle;

    // Signed contribution of edge a -> b; circle centered at the origin.
    static R edge(Point a, Point b, R r)
    {
        if (r == 0) return 0;
        Point v = b - a;
        R length = G::norm(v);
        if (length == 0) return 0;
        Point u = v / length;
        R h = G::cross(u, a);
        Point cuts[4] = {a};
        int count = 1;
        if (fabsl(h) <= r)
        {
            R delta = sqrtl(max(R(0), (r - fabsl(h)) * (r + fabsl(h))));
            R t = -G::dot(a, u);
            Point foot = G::perp(u) * h;
            for (R s : {-delta, delta})
                if (t + s > 0 && t + s < length) cuts[count++] = foot + u * s;
        }
        cuts[count++] = b;
```

分段面积与逐边累加（接上页同一结构体与函数）：

```
        R answer = 0;
        for (int i = 1; i < count; i++)
        {
            Point p = cuts[i - 1], q = cuts[i];
            R cross = G::cross(p, q);
            if (G::norm((p + q) / 2) <= r)
                answer += cross / 2;
            else
                answer += r * r * atan2l(cross, G::dot(p, q)) / 2;
        }
        return answer;
    }

    // Simple polygon, either orientation; duplicate consecutive vertices allowed.
    static R area(const vector<Point> &p, Circle c)
    {
        assert(c.r >= 0);
        if (p.size() < 3 || c.r == 0) return 0;
        R answer = 0, error = 0;
        for (int i = 0; i < (int)p.size(); i++)
        {
            R value = edge(p[i] - c.o, p[(i + 1) % p.size()] - c.o, c.r);
            R y = value - error;
            R sum = answer + y;
            error = (sum - answer) - y;
            answer = sum;
        }
        return fabsl(answer);
    }
};
```

模板题使用：AOJ CGL_7_H

题意：原点圆与逆时针简单多边形的公共面积，允许凹多边形。n 为 3..100，整数坐标绝对值 ≤100，半径 1..100。直接传入顶点序列，不取凸包；绝对误差小于 1e-5。

所需模板：RealPlane（第 1 页，1.1 节）、CirclePolygon（第 23 页，1.13 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.u-aizu.ac.jp/onlinejudge/description.jsp?id=CGL_7_H

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    long double r;
    cin >> n >> r;
    vector<RealPlane::Point> p(n);
    for (auto &a : p) cin >> a.x >> a.y;
    cout << fixed << setprecision(12) << CirclePolygon::area(p, {{0, 0}, r}) << '\n';
}
```

1.14 两圆公切线与过点切线

代码依赖：RealPlane（第 1 页，1.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

solve(a,b,eps) 返回 Result，包含 infinite 和 lines；infinite 为真表示无穷多条，否则 lines 给出全部切线。每条 Tangent 保存切点 a、b，单位法向量 normal，以及 side。a、b 对应输入两圆，直线方程 dot(normal,x-a)=0；切点相同时仍有效。side=1 外公切线、-1 内公切线、0 两个不同点。半径零表示点，from_point(p,c,eps) 等价于第一圆取点，圆上切点读 b。输入有限，坐标及半径绝对值 ≤1e100，半径非负。eps 默认 1e-12，用于相对圆心距的近相切判断；近相切可能合并为一条，浮点坐标仍有舍入误差。严格同心等半径时 infinite=true，包括同一点；同心不等半径返回空。O(1)；依赖浮点几何的 Point/Circle、点运算及 norm/perp。

```
struct CircleTangents
{
    using G = RealPlane;
    using R = G::R;
    using Point = G::Point;
    using Circle = G::Circle;

    struct Tangent
    {
        Point a, b, normal;
        int side; // 1 outer, -1 inner, 0 two distinct point circles.
    };

    struct Result
    {
        bool infinite = false;
        vector<Tangent> lines;
    };

    // Line: dot(normal, x - a) = 0. Radius zero represents a point.
    static Result solve(Circle a, Circle b, R eps = 1e-12L)
    {
        assert(a.r >= 0 && b.r >= 0 && eps > 0 && eps < 1);
        Point v = b.o - a.o;
        R d = G::norm(v);
        if (d == 0) return {a.r == b.r, {}};
        Point u = v / d;
        if (a.r == 0 && b.r == 0) return {false, {{a.o, b.o, G::perp(u), 0}}};
```

内外公切线枚举（接上页同一函数）：

```
        Result answer;
        for (int side : {1, -1})
        {
            if (side == -1 && (a.r == 0 || b.r == 0)) continue;
            R gap = a.r - side * b.r;
            if (fabsl(gap) > d * (1 + eps)) continue;
            bool touch = fabsl(gap) >= d * (1 - eps);
            R c = touch ? (gap < 0 ? -1 : 1) : gap / d;
            R s = sqrtl(max(R(0), (1 - c) * (1 + c)));
            for (int sign : {-1, 1})
            {
                Point normal = u * c + G::perp(u) * (sign * s);
                Point p = a.o + normal * a.r;
                Point q = b.o + normal * (side * b.r);
                answer.lines.push_back({p, q, normal, side});
                if (touch) break;
            }
        }
        return answer;
    }

    static Result from_point(Point p, Circle c, R eps = 1e-12L)
    {
        return solve({p, 0}, c, eps);
    }
};
```

模板题使用：AOJ CGL_7_F

题意：过圆外整数点作切线，输出圆上的两个切点。坐标绝对值 ≤1000，半径 1..1000，保证点严格在圆外。from_point 把点作为第一圆，圆上切点在 line.b；按 x/y 字典序输出，每行一个点，绝对误差小于 1e-5。

所需模板：RealPlane（第 1 页，1.1 节）、CircleTangents（第 25 页，1.14 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.u-aizu.ac.jp/onlinejudge/description.jsp?id=CGL_7_F

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    RealPlane::Point p;
    RealPlane::Circle c;
    cin >> p.x >> p.y >> c.o.x >> c.o.y >> c.r;
    auto ans = CircleTangents::from_point(p, c);
    vector<RealPlane::Point> points;
    for (auto line : ans.lines) points.push_back(line.b);
    sort(points.begin(),
         points.end(),
         [](auto a, auto b) { return tie(a.x, a.y) < tie(b.x, b.y); });
    cout << fixed << setprecision(12);
    for (auto a : points) cout << a.x << ' ' << a.y << '\n';
}
```

1.15 整数圆公切线与精确切点排序

代码依赖：IntegerPlane（第 14 页，1.8 节）、RealPlane（第 1 页，1.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

solve(a,r,b,s)：a、b 为 IntegerPlane::Point，所有坐标绝对值 $\le 10^4$，半径 $0..10^4$。依赖 RealPlane；返回 Result{infinite,lines}，每条 Tangent 含第一/第二圆切点 a、b、单位法向 normal、side（1 外切、-1 内切、0 两点圆）。直线满足 dot(normal,x-a)=0，不能用相切时重合的两个切点定直线。精确整数判断条数，并用根式符号精确比较第一圆切点的 (x,y) 次序；返回顺序已正确，不要按舍入后的坐标重排。同心同半径 infinite=true（含两个相同点圆），同心不同半径为空；异心两个点圆仅一条线。坐标和法向仍为 long double 近似构造。O(1) 时间/空间，最多 4 条。sign/root_sign/compare 仅为内部比较，不能任意扩大参数范围。此范围较圆交点版更小，因为排序包含二次平方；不是任意 64 位整数几何。

```
struct IntegerTangents
{
    using I = __int128_t;
    using R = RealPlane::R;
    using Point = IntegerPlane::Point;

    struct Tangent
    {
        RealPlane::Point a, b, normal;
        int side;
    };

    struct Result
    {
        bool infinite = false;
        vector<Tangent> lines;
    };

    static int sign(I x) { return (x > 0) - (x < 0); }

    // Internal: sign(a + b * sqrt(c)), c >= 0.
    static int root_sign(I a, I b, I c)
    {
        if (b == 0 || c == 0) return sign(a);
        if (a == 0 || sign(a) == sign(b)) return sign(b);
        return sign(a) * sign(a * a - b * b * c);
    }

    // Internal: sign(a + b * sqrt(c) + e * sqrt(f)).
    static int compare(I a, I b, I c, I e, I f)
    {
        int u = root_sign(a, b, c), v = f == 0 ? 0 : sign(e);
        if (u == 0) return v;
        if (v == 0 || u == v) return u;
        return u * root_sign(a * a + b * b * c - e * e * f, 2 * a * b, c);
    }
```

公切线枚举与精确切点排序（接上页同一结构体）：

```
    // |coordinates| <= 10000, 0 <= radii <= 10000.
    // Lines are ordered by their exact contact (x, y) on the first circle.
    static Result solve(Point a, long long r, Point b, long long s)
    {
        for (auto p : {a, b})
            assert(-10000 <= p.x && p.x <= 10000 && -10000 <= p.y && p.y <= 10000);
        assert(0 <= r && r <= 10000 && 0 <= s && s <= 10000);
        I x = I(b.x) - a.x, y = I(b.y) - a.y;
        I q = x * x + y * y;
        if (q == 0) return {r == s, {}};
        RealPlane::Point v{R(x), R(y)}, o{R(a.x), R(a.y)}, p{R(b.x), R(b.y)};
        if (r == 0 && s == 0)
            return {false, {{o, p, RealPlane::perp(v) / sqrtl(R(q)), 0}}};

        struct Item
        {
            I g, d;
            int sign, side;
        };

        vector<Item> items;
        for (int side : {1, -1})
        {
            if (side == -1 && (r == 0 || s == 0)) continue;
            I g = I(r) - side * s, d = q - g * g;
            if (d < 0) continue;
            items.push_back({g, d, -1, side});
            if (d > 0) items.push_back({g, d, 1, side});
        }
        sort(items.begin(),
             items.end(),
             [&](Item a, Item b)
             {
                 int k = compare(x * (a.g - b.g), -y * a.sign, a.d, y * b.sign, b.d);
                 if (k != 0) return k < 0;
                 return compare(y * (a.g - b.g), x * a.sign, a.d, -x * b.sign, b.d) < 0;
             });
        Result answer;
        for (auto t : items)
        {
            auto n =
                (v * R(t.g) + RealPlane::perp(v) * (t.sign * sqrtl(R(t.d)))) / R(q);
            answer.lines.push_back({o + n * R(r), p + n * R(t.side * s), n, t.side});
        }
        return answer;
    }
};
```

模板题使用：AOJ CGL_7_G

题意：不同整数圆的公切线，输出第一圆上的全部切点。坐标绝对值 ≤1000，半径 1..1000；可能 0..4 条，同心不等圆需空输出。整数版已精确排序，直接顺序输出 line.a，不再按舍入值排序；绝对误差小于 1e-5。

所需模板：IntegerPlane（第 14 页，1.8 节）、RealPlane（第 1 页，1.1 节）、IntegerTangents（第 27 页，1.15 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.u-aizu.ac.jp/onlinejudge/description.jsp?id=CGL_7_G

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    IntegerPlane::Point a, b;
    long long r, s;
    cin >> a.x >> a.y >> r >> b.x >> b.y >> s;
    auto ans = IntegerTangents::solve(a, r, b, s);
    cout << fixed << setprecision(12);
    for (auto line : ans.lines) cout << line.a.x << ' ' << line.a.y << '\n';
}
```

相关应用：任意简单整数多边形面积并；符号化右侧极限、共线边编号去重和有理参数排序，浮点面积输出。

1.16 一般多边形面积并：整数精确判定

代码依赖：IntegerPlane（第 14 页，1.8 节）。抄写时一并准备；标准库类型不列入算法依赖。

polygon_union_area(polygons) 求任意简单多边形的面积并，允许凹多边形、不同多边形边界交叉/重合以及并集孔洞；每个输入多边形本身不含洞、不自交。整数顶点为 IntegerPlane::Point，坐标绝对值 $\leq 10^9$（比基础点类型约定更严格）。自动转逆时针，删除连续重复点及末尾重复首点；零面积退化输入忽略，但不负责检查自交。

共 V 个顶点，时间 $O(V^2 \log V)$、空间 O(V)，下标/覆盖数须可用 int 表示。128 位叉积与有理参数排序精确处理拓扑，最终面积 long double 近似；不扰动顶点、不用 eps 排序。包围盒相交的多边形共用局部原点并补偿求和，仍须按题目精度检验最终面积，不声称浮点累加无条件精确。

```
long double polygon_union_area(vector<vector<IntegerPlane::Point>> polygons)
{
    using G = IntegerPlane;
    using P = G::Point;
    using I = G::I;
    using R = long double;
    struct Fraction
    {
        I num, den;
        Fraction(I x = 0, I y = 1) : num(x), den(y)
        {
            assert(den != 0);
            if (den < 0)
            {
                num = -num;
                den = -den;
            }
        }
        bool operator<(const Fraction &b) const
        {
            return num * b.den < b.num * den;
        }
    };
    vector<vector<P>> a;
    for (auto p : polygons)
    {
        for (auto v : p)
            assert(-1000000000 <= v.x && v.x <= 1000000000 &&
                   -1000000000 <= v.y && v.y <= 1000000000);
        p.erase(unique(p.begin(), p.end()), p.end());
        if (p.size() > 1 && p.front() == p.back()) p.pop_back();
        I area = 0;
        int m = p.size();
        for (int i = 0; i < m; i++) area += G::cross({0, 0}, p[i], p[(i + 1) % m]);
        if (area == 0) continue;
        if (area < 0) reverse(p.begin(), p.end());
        a.push_back(move(p));
    }
```

局部原点与边事件初始化（接上页函数）：

```
    int n = a.size();
    vector<array<long long, 4>> box(n);
    vector<int> parent(n);
    iota(parent.begin(), parent.end(), 0);
    auto find = [&](auto &&self, int x) -> int
    {
        if (parent[x] == x) return x;
        return parent[x] = self(self, parent[x]);
    };
    for (int i = 0; i < n; i++)
    {
        auto p = a[i][0];
        box[i] = {p.x, p.y, p.x, p.y};
        for (auto v : a[i])
        {
            box[i][0] = min(box[i][0], v.x);
            box[i][1] = min(box[i][1], v.y);
            box[i][2] = max(box[i][2], v.x);
            box[i][3] = max(box[i][3], v.y);
        }
        for (int j = 0; j < i; j++)
            if (max(box[i][0], box[j][0]) <= min(box[i][2], box[j][2]) &&
                max(box[i][1], box[j][1]) <= min(box[i][3], box[j][3]))
                parent[find(find, i)] = find(find, j);
    }
    R answer = 0;
    R error = 0;
    for (int i = 0; i < n; i++)
    {
        int m = a[i].size();
        P origin = a[find(find, i)][0];
        for (int k = 0; k < m; k++)
        {
            P p = a[i][k];
            P q = a[i][(k + 1) % m];
            vector<pair<Fraction, int>> events;
            auto add = [&](Fraction t, int delta)
            {
                t = max(Fraction(0), min(Fraction(1), t));
                events.push_back({t, delta});
            };
```

右侧覆盖与共线边去重（接上页函数）：

```
        for (int j = 0; j < n; j++)
        {
            if (i == j) continue;
            int size = a[j].size();
            for (int h = 0; h < size; h++)
            {
                P c = a[j][h];
                P d = a[j][(h + 1) % size];
                I x = G::cross(p, q, c);
                I y = G::cross(p, q, d);
                if ((x >= 0) != (y >= 0))
                    add(Fraction(G::cross(c, d, p), y - x), x < 0 ? -1 : 1);
                if (x == 0 && y == 0 && j < i)
                {
                    auto project = [&](P v)
                    {
                        if (p.x != q.x) return Fraction(I(v.x) - p.x, I(q.x) - p.x);
                        return Fraction(I(v.y) - p.y, I(q.y) - p.y);
                    };
                    auto l = project(c);
                    auto r = project(d);
                    if (l < r)
                    {
                        add(l, 1);
                        add(r, -1);
                    }
                }
            }
        }
```

有理参数排序及未覆盖段积分（接上页函数）：

```
            add(Fraction(1), 0);
            sort(events.begin(), events.end(), [](const auto &x, const auto &y)
            {
                return x.first < y.first;
            });
            int cover = 0;
            Fraction last;
            for (auto [t, delta] : events)
            {
                if (cover == 0)
                {
                    I num = t.num * last.den - last.num * t.den;
                    I den = t.den * last.den;
                    R value = R(G::cross(origin, p, q)) * (R(num) / R(den)) / 2;
                    R y = value - error;
                    R next = answer + y;
                    error = (next - answer) - y;
                    answer = next;
                }
                cover += delta;
                last = t;
            }
        }
    }
    return answer;
}
```

精确范围 令坐标界 M=10 的 9 次方。差向量分量至多 2M，叉积、交点参数分子及分母绝对值至多 8M 平方。分式比较的单个交叉乘积至多 64M 四次方，小于 128 位有符号上界；裁到 [0,1] 后的参数差也安全。不能直接沿用 IntegerPlane 允许的更大坐标界。

为何看边的右侧 每个多边形先转成逆时针，当前边的左侧已在并集中。只有其他多边形占据右侧时，该边才成为并集内部，不能贡献边界。把测试线向右侧平移无穷小量，原线上顶点视作在线上方：cross 大于等于 0 对比小于 0，等价于符号化极限，没有实际扰动坐标。向下穿越时覆盖加 1，向上穿越时减 1。凹多边形会产生多个区间，仍由相同事件累计处理。

共享边与端点 两条共线反向边的内部位于两侧，右侧测试已将其剔除；同向共线边仅编号较小的多边形保留，避免重复积分。所有事件参数精确裁到 [0,1]，同参数事件的任意顺序只影响零长度段，不影响面积。退化顶点、相切和部分重叠边均不需要随机扰动。

积分与孔洞 对边 p 到 q 的未覆盖参数段 [l,u]，贡献是 cross(p-origin,q-origin) 乘 (u-l)/2。按逆时针原边方向累加，孔洞自动带负方向，不对单边取绝对值。包围盒相交关系的连通块包含真实并集连通块，按该分组平移不改变闭合积分。最终转 long double 和补偿求和仍有误差；输入内含洞或自交轮廓不在本接口范围。

独立验证 Fraction 精确枚举所有线段交点横坐标，分竖直条带合并各多边形截面，以线性上下包络的梯形公式求真值。包含凹多边形、重复/共线/反向边、孔洞、极值坐标、刚体变换和 1000 三角形压力例。线上状态见验证记录。

模板题使用：P13338 三角形面积并加强版

题意：n≤1000 个非退化三角形，整数坐标绝对值 $\leq 10^6$；误差要求 | 答案-真值 |/max(1,| 真值 |)≤1e-6。此驱动使用一般多边形模板；凹多边形能力另由本地验证覆盖。当前实现已通过 P13338，记录 302630852。

所需模板：IntegerPlane（第 14 页，1.8 节）、polygon_union_area（第 30 页，1.16 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P13338

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<vector<IntegerPlane::Point>> a(n, vector<IntegerPlane::Point>(3));
    for (auto &p : a)
        for (auto &v : p) cin >> v.x >> v.y;
    cout << fixed << setprecision(12) << polygon_union_area(a) << '\n';
}
```

相关应用：圆面积并的圆弧覆盖与格林公式；支持包含、同圆、相切及孔洞，明确浮点局限。

1.17 圆面积并：未覆盖圆弧积分

代码依赖：RealPlane（第 1 页，1.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

circle_union_area(a) 求闭圆盘的面积并，输入 vector<RealPlane::Circle>，圆心及半径用 long double，半径 ≥0；空集/点圆贡献 0。同圆只保留最早编号的边界，内含圆无贡献；不要求连通或无孔洞。相交连通块各自平移后积分，原输入不改变。

每个圆将其他圆覆盖的角区间切到 [0,2π]，严格排序合并，只积分未覆盖的圆弧。$O(n^2 \log n)$ 时间、O(n) 额外空间；并查集递归最深 O(n)。所有输入和中间运算须有限可表示。使用原始浮点比较，不用 eps 排序或随机扰动；近退化分类、极端尺度和最终误差仍受浮点限制。整数题域与本地对拍不构成任意实数的精确保证。

```
RealPlane::R circle_union_area(vector<RealPlane::Circle> a)
{
    using R = RealPlane::R;
    R pi = acosl(-1.L);
    R tau = 2 * pi;
    for (auto c : a)
        assert(isfinite(c.o.x) && isfinite(c.o.y) && isfinite(c.r) && c.r >= 0);
    a.erase(remove_if(a.begin(), a.end(), [](auto c)
    {
        return c.r == 0;
    }), a.end());
    int n = a.size();
    vector<int> parent(n);
    iota(parent.begin(), parent.end(), 0);
    auto find = [&](auto &&self, int x) -> int
    {
        if (parent[x] == x) return x;
        return parent[x] = self(self, parent[x]);
    };
    for (int i = 0; i < n; i++)
        for (int j = 0; j < i; j++)
            if (RealPlane::norm(a[i].o - a[j].o) < a[i].r + a[j].r)
                parent[find(find, i)] = find(find, j);
    R answer = 0;
    R error = 0;
    for (int i = 0; i < n; i++)
    {
        auto c = a[i];
        auto o = c.o - a[find(find, i)].o;
        vector<pair<R, R>> intervals;
        bool covered = false;
```

两圆关系及覆盖角区间（接上页函数）：

```
        for (int j = 0; j < n; j++)
        {
            if (i == j) continue;
            auto v = a[j].o - c.o;
            R d = RealPlane::norm(v);
            R r = c.r;
            R s = a[j].r;
            if (s >= r && d <= s - r && (s > r || j < i))
            {
                covered = true;
                break;
            }
            if (d >= r + s || d <= abs(r - s)) continue;
            R scale = max({d, r, s});
            d /= scale;
            r /= scale;
            s /= scale;
            R h = (r + s - d) * (d + r - s) * (d - r + s) * (d + r + s);
            R angle = atan2l(sqrtl(max(R(0), h)), d * d + (r - s) * (r + s));
            R middle = atan2l(v.y, v.x);
            if (middle < 0) middle += tau;
            R l = middle - angle;
            R u = middle + angle;
            if (l < 0)
            {
                intervals.push_back({0, u});
                intervals.push_back({l + tau, tau});
            }
            else if (u > tau)
            {
                intervals.push_back({l, tau});
                intervals.push_back({0, u - tau});
            }
            else intervals.push_back({l, u});
        }
        if (covered) continue;
```

未覆盖圆弧积分与补偿求和（接上页函数）：

```
        auto add = [&](R l, R r)
        {
            R h = (r - l) / 2;
            R middle = (r + l) / 2;
            R value = c.r * c.r * h;
            if (r - l != tau)
                value += c.r * sinl(h) * (o.x * cosl(middle) + o.y * sinl(middle));
            R y = value - error;
            R next = answer + y;
            error = (next - answer) - y;
            answer = next;
        };
        sort(intervals.begin(), intervals.end());
        R right = 0;
        for (auto [l, r] : intervals)
        {
            if (l > right) add(right, l);
            right = max(right, r);
        }
        if (right < tau) add(right, tau);
    }
    return answer;
}
```

为什么只算未覆盖弧 删去重合边界后，圆盘并边界恰是没有落在其他圆内部的圆弧。各圆逆时针方向的内部位于左侧，所以外边界和孔洞边界自动获得正确符号；不能对每条弧单独取绝对值。相切点面积为零，内含圆无需贡献。

积分公式 对相对连通块原点的圆心 (x, y)，半径 r，弧角从 l 到 u，令 $h = (u - l)/2$、$t = (u + l)/2$。格林公式给出贡献 $r^2 h + r \sin h(x \cos t + y \sin t)$。整圆直接用 πr^2，避免 $\sin \pi$ 舍入误差；补偿求和减少大量正负弧贡献相加的损失。连通块分别平移不改变闭合边界积分。

数值与验证 角度排序使用严格原值比较，同圆按编号去重，不以 eps 合并不同圆。半角通过缩放后的 Heron 乘积与 atan2 求出，避免 acos 靠近正负 1 时的敏感性；仍不是精确几何。独立 70 位高精度验证沿竖直截面、在全部交点横坐标处分段解析积分，另有孔洞、相切、远平移和千圆压力例。线上 AC 状态见验证记录，不能由本地通过推断。

模板题使用：SP8073 CIRU 圆面积并

题意：n≤1000，整数圆心坐标绝对值及半径 ≤1000，允许零半径。输出圆盘面积并，保留三位小数；SPOJ 转交记录 302630657 仍 Waiting。当前核心另已通过用户模板题 U106005，输出精度适配见下一用法。

所需模板：RealPlane（第 1 页，1.1 节）、circle_union_area（第 36 页，1.17 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/SP8073

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<RealPlane::Circle> circles(n);
    for (auto &c : circles) cin >> c.o.x >> c.o.y >> c.r;
    cout << fixed << setprecision(3) << circle_union_area(circles) << '\n';
}
```

模板题使用：U106005 圆面积并（用户模板题）

题意：n≤1000，整数圆心坐标绝对值及半径 ≤1000，允许零半径。误差要求 1e-3，SPJ 要求至少 4 位小数，故输出 12 位。当前源码记录 302633427，5 点全 AC；不代表 SPOJ 转交记录已通过。

所需模板：RealPlane（第 1 页，1.1 节）、circle_union_area（第 36 页，1.17 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/U106005

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<RealPlane::Circle> circles(n);
    for (auto &c : circles) cin >> c.o.x >> c.o.y >> c.r;
    cout << fixed << setprecision(12) << circle_union_area(circles) << '\n';
}
```

相关应用：分层圆弧积分求每个精确覆盖重数的面积，重复圆按编号分层；有独立高精度截面积分验证。

1.18 圆覆盖重数面积：恰好覆盖 k 次

代码依赖：RealPlane（第 1 页，1.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

circle_coverage_area(a) 返回长 n+1 的数组，ans[k] 为恰被 k 个闭圆盘覆盖的面积（1≤k≤n）；ans[0]=0 仅为占位，不表示无界平面面积。输入 vector<RealPlane::Circle>，圆心和半径 long double，半径 ≥0，n≤INT_MAX-2；按值传参不改原数组。支持重复圆、零半径、包含、相切和孔洞。

圆弧按覆盖层数积分得到至少 k 次面积，再相邻作差；同圆按编号依次归入不同层，不能去重。$O(n^2 \log n)$ 时间、O(n) 额外空间，并查集递归最深 O(n)。输入和中间运算须有限可表示；严格原值排序、分块平移及补偿求和不能消除近退化与小面积差分的浮点误差。不对核心结果强制取绝对值或截零。

```
vector<RealPlane::R> circle_coverage_area(vector<RealPlane::Circle> a)
{
    using R = RealPlane::R;
    R pi = acosl(-1.L);
    R tau = 2 * pi;
    for (auto c : a)
        assert(isfinite(c.o.x) && isfinite(c.o.y) && isfinite(c.r) && c.r >= 0);
    assert(a.size() <= INT_MAX - 2);
    int size = int(a.size());
    a.erase(remove_if(a.begin(), a.end(), [](auto c)
    {
        return c.r == 0;
    }), a.end());
    int n = int(a.size());
    vector<int> parent(n);
    iota(parent.begin(), parent.end(), 0);
    auto find = [&](auto &&self, int x) -> int
    {
        if (parent[x] == x)
            return x;
        return parent[x] = self(self, parent[x]);
    };
    for (int i = 0; i < n; i++)
        for (int j = 0; j < i; j++)
            if (RealPlane::norm(a[i].o - a[j].o) < a[i].r + a[j].r)
                parent[find(find, i)] = find(find, j);
    vector<R> answer(size + 2);
    vector<R> error(size + 2);
    for (int i = 0; i < n; i++)
    {
        auto c = a[i];
        auto o = c.o - a[find(find, i)].o;
        vector<pair<R, int>> events = {{0, 0}, {tau, 0}};
        int count = 1;
```

两圆关系及覆盖角区间（接上页函数）：

```
        for (int j = 0; j < n; j++)
        {
            if (i == j)
                continue;
            auto v = a[j].o - c.o;
            R d = RealPlane::norm(v);
            R r = c.r;
            R s = a[j].r;
            if (s >= r && d <= s - r && (s > r || j < i))
            {
                count++;
                continue;
            }
            if (d >= r + s || d <= abs(r - s))
                continue;
            R scale = max({d, r, s});
            d /= scale;
            r /= scale;
            s /= scale;
            R h = (r + s - d) * (d + r - s) * (d - r + s) * (d + r + s);
            R angle = atan2l(sqrtl(max(R(0), h)), d * d + (r - s) * (r + s));
            R middle = atan2l(v.y, v.x);
            if (middle < 0)
                middle += tau;
            R l = middle - angle;
            R u = middle + angle;
            if (l < 0)
            {
                events.push_back({0, 1});
                events.push_back({u, -1});
                events.push_back({l + tau, 1});
                events.push_back({tau, -1});
            }
            else if (u > tau)
            {
                events.push_back({l, 1});
                events.push_back({tau, -1});
                events.push_back({0, 1});
                events.push_back({u - tau, -1});
            }
            else
            {
                events.push_back({l, 1});
                events.push_back({u, -1});
            }
        }
```

分层圆弧积分与补偿求和（接上页函数）：

```
        auto add = [&](R l, R r)
        {
            R h = (r - l) / 2;
            R middle = (r + l) / 2;
            R value = c.r * c.r * h;
            if (r - l != tau)
                value += c.r * sinl(h) * (o.x * cosl(middle) + o.y * sinl(middle));
            R y = value - error[count];
            R next = answer[count] + y;
            error[count] = (next - answer[count]) - y;
            answer[count] = next;
        };
        sort(events.begin(), events.end());
        R left = 0;
        for (auto [right, delta] : events)
        {
            if (right > left)
                add(left, right);
            count += delta;
            left = right;
        }
    }
    for (int k = 1; k <= size; k++)
        answer[k] -= answer[k + 1];
    answer.resize(size + 1);
    return answer;
}
```

覆盖分层 当前圆弧左侧包含自身，另有 count-1 个圆覆盖；它是至少 count 次覆盖区域的正向边界。分别积分到对应层后，按 k 递增作 answer[k]-answer[k+1] 得到恰好 k 次。重合的 m 个圆按编号让第 t 个额外计入前 t-1 个同圆，从而每个阈值恰贡献一次边界；不能去重。ans[0] 只是占位。

数值与验证 各连通块平移、严格角排序、补偿累加；仍有浮点分类和相邻层相减的消去误差，小面积可能有微小负误差。466 组 70 位竖条截面积分及解析压力例，在 O2、NDEBUG、抄写版与 ASan/UBSan 下通过；不据此承诺任意实数精确。

积分公式 对相对连通块原点的圆心 (x, y)，半径 r，弧角从 l 到 u，令 $h = (u - l)/2$、$t = (u + l)/2$。格林公式给出贡献 $r^2 h + r \sin h(x \cos t + y \sin t)$。整圆直接用 πr^2，避免 $\sin \pi$ 舍入误差；补偿求和减少大量正负弧贡献相加的损失。连通块分别平移不改变闭合边界积分。

模板题使用：SP8119 CIRUT 圆覆盖重数面积

题意：n≤1000，互异圆，整数圆心坐标绝对值 ≤1000、0<r≤1000；输出恰被 k 个圆覆盖的面积，保留三位小数。核心额外支持同圆及零半径，另由本地独立验证覆盖。末端 max 仅避免舍入负零，不改变核心结果。已提交记录 302660709，仍 Waiting，不记 AC。

所需模板：RealPlane（第 1 页，1.1 节）、circle_coverage_area（第 40 页，1.18 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/SP8119

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<RealPlane::Circle> circles(n);
    for (auto &c : circles)
        cin >> c.o.x >> c.o.y >> c.r;
    auto area = circle_coverage_area(circles);
    cout << fixed << setprecision(3);
    for (int k = 1; k <= n; k++)
        cout << '[' << k << "] = " << max(0.L, area[k]) << '\n';
}
```

第二章　三维计算几何基础

2.1　整数三维共线、共面与方向

坐标绝对值 $\le 10^9$；orient(a,b,c,d) 为有向六倍四面体体积，0 表示共面；collinear 判三点共线，on_segment 支持退化线段。int128 精确判定，不包含三维凸包构造。

```
struct IntegerGeometry3D
{
    using I = __int128_t;

    // |coordinates|<=1e9: triple products fit signed int128.
    struct Point
    {
        long long x, y, z;
    };

    struct Vector
    {
        I x, y, z;
    };

    static Vector diff(Point a, Point b)
    {
        return {I(a.x) - b.x, I(a.y) - b.y, I(a.z) - b.z};
    }

    static Vector cross(Vector a, Vector b)
    {
        return {a.y * b.z - a.z * b.y, a.z * b.x - a.x * b.z, a.x * b.y - a.y * b.x};
    }

    static I dot(Vector a, Vector b) { return a.x * b.x + a.y * b.y + a.z * b.z; }

    static I orient(Point a, Point b, Point c, Point d)
    {
        return dot(cross(diff(b, a), diff(c, a)), diff(d, a));
    }

    static bool collinear(Point a, Point b, Point c)
    {
        auto v = cross(diff(b, a), diff(c, a));
        return !v.x && !v.y && !v.z;
    }

    static bool on_segment(Point p, Point a, Point b)
    {
        return collinear(p, a, b) && dot(diff(a, p), diff(b, p)) <= 0;
    }
};
```

接口用法演示：自定义整数三维谓词

演示约定：自定义协议，非 OJ 题：0≤Q≤200000，每次给 a,b,c,d 四个整数点，各坐标绝对值 ≤10^9。输出 orient(a,b,c,d)、collinear(a,b,c)、on_segment(d,a,b)，后两项为 0/1。第一项是有向六倍体积，用 128 位整数精确输出，不能整除 6 后冒充体积；为 0 也不保证前三点确定唯一平面。允许重复点、退化和闭线段端点。此点域的安全界不扩展到任意 Vector 参数或反复组合运算。

所需模板：IntegerGeometry3D（第 44 页，2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://github.com/gsh20040816/CPCtemplate/blob/main/src/compact/geometry_extra.hpp

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    auto print = [](__int128_t x)
    {
        // Unsigned subtraction also handles the minimum signed int128 value.
        __uint128_t u = x < 0 ? __uint128_t(0) - __uint128_t(x) : __uint128_t(x);
        string s;
        do { s += char('0' + u % 10); u /= 10; } while (u);
        if (x < 0) cout << '-';
        reverse(s.begin(), s.end());
        cout << s;
    };
    int q;
    if (!(cin >> q)) return 0;
    while (q--)
    {
        IntegerGeometry3D::Point a, b, c, d;
        for (auto p : {&a, &b, &c, &d}) cin >> p->x >> p->y >> p->z;
        print(IntegerGeometry3D::orient(a, b, c, d));
        cout << ' ' << IntegerGeometry3D::collinear(a, b, c)
             << ' ' << IntegerGeometry3D::on_segment(d, a, b) << '\n';
    }
}
```

相关应用：不扰动增量凸包，精确方向符号、低维退化输出、面积及六倍体积。

2.2 三维凸包：精确方向与退化处理

Hull3D<T>(points)，T 只取 long long 或 long double（默认）。输入编号从 0 起，不修改原数组。p 为去重后字典序点，id 映射首次原编号；faces 是 p 下标组成的外向三角面。dimension 为空/点/线/平面/三维时分别为-1/0/1/2/3；低维 boundary 给出点、线端点或平面凸多边形边界。area() 在平面时只算单面，三维时算表面积；volume6() 返回六倍体积，低维为 0。

整数坐标绝对值 $\leq 10^9$，方向与六倍体积用 128 位精确运算。实数用过滤后浮点展开式确定符号，不用 eps、不扰动坐标；按已读入的二进制数求凸包，面积和实数体积仍为近似。需最近舍入、禁用 fast-math，运算及残差不得上下溢；足够安全的坐标域和具体约定见后页。

时间 $O(n^2 \log n)$、额外空间 $O(n)$。外露字段用于读取结果，不应构造后自行改写。点数与面下标须能用 int 表示。

字段与面积运算：

```
template <class T = long double>
struct Hull3D
{
    static_assert(is_same_v<T, long long> || is_same_v<T, long double>);
    using W = conditional_t<is_same_v<T, long long>, __int128, long double>;
    struct Point
    {
        T x, y, z;
    };
    vector<Point> p;
    vector<int> id, boundary;
    vector<array<int, 3>> faces;
    int dimension = -1;

    static array<W, 3> normal(Point a, Point b, Point c)
    {
        W x = W(b.x) - a.x;
        W y = W(b.y) - a.y;
        W z = W(b.z) - a.z;
        W u = W(c.x) - a.x;
        W v = W(c.y) - a.y;
        W w = W(c.z) - a.z;
        return {y * w - z * v, z * u - x * w, x * v - y * u};
    }
    static W orient(Point a, Point b, Point c, Point d)
    {
        auto v = normal(a, b, c);
        return v[0] * (W(d.x) - a.x) + v[1] * (W(d.y) - a.y) +
               v[2] * (W(d.z) - a.z);
    }
```

浮点展开式运算（接同一结构体）：

```
// Exact signs for represented floating inputs; nearest rounding, no fast-math.
// Products and residuals must neither overflow nor underflow.
using Expansion = vector<long double>;
static void grow(Expansion &a, long double q)
{
    Expansion out;
    for (long double x : a)
    {
        long double s = q + x;
        long double v = s - q;
        long double error = (q - (s - v)) + (x - v);
        if (error != 0) out.push_back(error);
        q = s;
    }
    if (q != 0) out.push_back(q);
    a.swap(out);
}
static Expansion difference(long double a, long double b)
{
    Expansion result{a};
    grow(result, -b);
    return result;
}
static Expansion product(const Expansion &a, const Expansion &b)
{
    Expansion result;
    for (long double x : a)
        for (long double y : b)
        {
            long double value = x * y;
            grow(result, fmal(x, y, -value));
            grow(result, value);
        }
    return result;
}
static int expansion_sign(const Expansion &a)
{
    if (a.empty()) return 0;
    return a.back() > 0 ? 1 : -1;
}
```

二维方向与共线检查（接同一结构体）：

```
    static int turn_sign(T x, T y, T u, T v, T s, T t)
    {
        W a = W(u) - x;
        W b = W(v) - y;
        W c = W(s) - x;
        W d = W(t) - y;
        W value = a * d - b * c;
        if constexpr (is_same_v<T, long long>)
            return (value > 0) - (value < 0);
        else
        {
            long double bound = 32 * numeric_limits<long double>::epsilon() *
                                (abs(a * d) + abs(b * c));
            if (abs(value) > bound) return value > 0 ? 1 : -1;
            auto result = product(difference(u, x), difference(t, y));
            auto subtract = product(difference(v, y), difference(s, x));
            for (long double z : subtract) grow(result, -z);
            return expansion_sign(result);
        }
    }
    static array<int, 3> normal_sign(Point a, Point b, Point c)
    {
        return {turn_sign(a.y, a.z, b.y, b.z, c.y, c.z),
                turn_sign(a.z, a.x, b.z, b.x, c.z, c.x),
                turn_sign(a.x, a.y, b.x, b.y, c.x, c.y)};
    }
```

三维方向的过滤与精确回退（接同一结构体）：

```
static int orient_sign(Point a, Point b, Point c, Point d)
{
    W value = orient(a, b, c, d);
    if constexpr (is_same_v<T, long long>)
        return (value > 0) - (value < 0);
    else
    {
        array<long double, 3> x{b.x - a.x, b.y - a.y, b.z - a.z};
        array<long double, 3> y{c.x - a.x, c.y - a.y, c.z - a.z};
        array<long double, 3> z{d.x - a.x, d.y - a.y, d.z - a.z};
        long double permanent = 0;
        for (int i = 0; i < 3; i++)
        {
            int j = (i + 1) % 3;
            int k = (i + 2) % 3;
            permanent += (abs(x[j] * y[k]) + abs(x[k] * y[j])) * abs(z[i]);
        }
        long double bound = 128 * numeric_limits<long double>::epsilon() * permanent;
        if (abs(value) > bound) return value > 0 ? 1 : -1;
        array<Expansion, 3> ex{difference(b.x, a.x), difference(b.y, a.y), difference(b.z, a.z)};
        array<Expansion, 3> ey{difference(c.x, a.x), difference(c.y, a.y), difference(c.z, a.z)};
        array<Expansion, 3> ez{difference(d.x, a.x), difference(d.y, a.y), difference(d.z, a.z)};
        Expansion result;
        for (int i = 0; i < 3; i++)
        {
            int j = (i + 1) % 3;
            int k = (i + 2) % 3;
            auto positive = product(product(ex[j], ey[k]), ez[i]);
            auto negative = product(product(ex[k], ey[j]), ez[i]);
            for (long double v : positive) grow(result, v);
            for (long double v : negative) grow(result, -v);
        }
        return expansion_sign(result);
    }
}
```

构造、排序与去重（接同一结构体）：

```
    explicit Hull3D(const vector<Point> &input)
    {
        vector<int> order(input.size());
        iota(order.begin(), order.end(), 0);
        for (auto v : input)
        {
            if constexpr (is_same_v<T, long long>)
                assert(-1000000000 <= v.x && v.x <= 1000000000 &&
                       -1000000000 <= v.y && v.y <= 1000000000 &&
                       -1000000000 <= v.z && v.z <= 1000000000);
            else assert(isfinite(v.x) && isfinite(v.y) && isfinite(v.z));
        }
        auto key = [](Point v)
        {
            return make_tuple(v.x, v.y, v.z);
        };
        sort(order.begin(), order.end(), [&](int a, int b)
        {
            return make_tuple(input[a].x, input[a].y, input[a].z, a) <
                   make_tuple(input[b].x, input[b].y, input[b].z, b);
        });
        for (int i : order)
            if (p.empty() || key(input[i]) != key(p.back()))
            {
                p.push_back(input[i]);
                id.push_back(i);
            }
```

低维识别与平面投影（接同一构造函数）：

```
    int n = p.size();
    if (n == 0) return;
    dimension = 0;
    boundary = {0};
    if (n == 1) return;
    dimension = 1;
    boundary = {0, n - 1};
    int c = 2;
    while (c < n && normal_sign(p[0], p[1], p[c]) == array<int, 3>{}) c++;
    if (c == n) return;
    auto axis = normal_sign(p[0], p[1], p[c]);
    int d = 2;
    while (d < n && orient_sign(p[0], p[1], p[c], p[d]) == 0) d++;
    if (d == n)
    {
        dimension = 2;
        int drop = 0;
        while (axis[drop] == 0) drop++;
        auto project = [&](Point v)
        {
            if (drop == 0) return pair<T, T>{v.y, v.z};
            if (drop == 1) return pair<T, T>{v.x, v.z};
            return pair<T, T>{v.x, v.y};
        };
        auto turn = [&](int a, int b, int c)
        {
            auto [x, y] = project(p[a]);
            auto [u, v] = project(p[b]);
            auto [s, t] = project(p[c]);
            return turn_sign(x, y, u, v, s, t);
        };
```

平面单调链与三角化（接同一构造函数）：

```
        order.resize(n);
        iota(order.begin(), order.end(), 0);
        sort(order.begin(), order.end(), [&](int a, int b)
        {
            return project(p[a]) < project(p[b]);
        });
        boundary.clear();
        for (int pass = 0; pass < 2; pass++)
        {
            int base = boundary.size();
            for (int v : order)
            {
                while ((int)boundary.size() >= base + 2 &&
                       turn(boundary[boundary.size() - 2], boundary.back(), v) <= 0)
                    boundary.pop_back();
                boundary.push_back(v);
            }
            boundary.pop_back();
            reverse(order.begin(), order.end());
        }
        for (int i = 1; i + 1 < (int)boundary.size(); i++)
            faces.push_back({boundary[0], boundary[i], boundary[i + 1]});
        return;
    }
```

初始四面体与增量可见面（接同一构造函数）：

```
        dimension = 3;
        boundary.clear();
        array<int, 4> seed{0, 1, c, d};
        for (int i = 0; i < 4; i++)
        {
            array<int, 3> f{seed[(i + 1) % 4], seed[(i + 2) % 4], seed[(i + 3) % 4]};
            if (orient_sign(p[f[0]], p[f[1]], p[f[2]], p[seed[i]]) > 0)
                swap(f[1], f[2]);
            faces.push_back(f);
        }
        for (int v = 0; v < n; v++)
        {
            if (find(seed.begin(), seed.end(), v) != seed.end()) continue;
            vector<array<int, 3>> keep;
            set<pair<int, int>> horizon;
            for (auto f : faces)
            {
                if (orient_sign(p[f[0]], p[f[1]], p[f[2]], p[v]) <= 0)
                {
                    keep.push_back(f);
                    continue;
                }
                for (int k = 0; k < 3; k++)
                {
                    int a = f[k];
                    int b = f[(k + 1) % 3];
                    if (!horizon.erase({b, a})) horizon.insert({a, b});
                }
            }
            for (auto [a, b] : horizon) keep.push_back({a, b, v});
            faces.swap(keep);
        }
    }
```

面积与六倍体积（接同一结构体）：

```
    long double area() const
    {
        long double answer = 0;
        for (auto f : faces)
        {
            auto v = normal(p[f[0]], p[f[1]], p[f[2]]);
            answer += hypotl(hypotl((long double)v[0], (long double)v[1]),
                             (long double)v[2]) / 2;
        }
        return answer;
    }
    W volume6() const
    {
        W answer = 0;
        if (dimension != 3) return answer;
        for (auto f : faces) answer += orient(p[0], p[f[0]], p[f[1]], p[f[2]]);
        return answer;
    }
};
```

不变量 三维初始四面体各面均背向内部。新增点只删除严格可见的面，删除面中的反向公共边两两消去，剩余有向边是轮廓。用轮廓边和新点组成新面即可恢复闭合外向凸包；零方向不算可见，不能扰动共面点。所有点先排序去重，平面情况投影到法向非零的轴，单独用二维凸包处理。

精确符号的范围 浮点展开式保存每次加法和乘法被舍弃的残差，末项符号即精确和的符号；乘法残差通过 fma 计算。先用保守舍入误差界过滤明显非零的方向，不能确定时回退。需二进制、最近舍入、禁用 fast-math 及运算重排，保证运算和残差不上下溢。对于有效位不超过 113 且指数范围至少覆盖普通 double 的实现，每个非零坐标绝对值在 2 的负 100 次方至 2 的 100 次方内是一个足够安全域。输入十进制读取误差不被恢复。

测量和退化约定 精确方向不等于精确面积。面积的平方根与求和、实数体积仍可能失去相对精度，极薄模型需额外检验。整数域内六倍体积精确且非负；平面核心只返回单面面积，点和线面积为 0。题目驱动选择平面双面包裹约定，题面本身没有明确低维语义；其他题应按题意适配。

独立验证 565 组输入分别验证整数和实数版，参考为精确支撑平面枚举及每个面的二维凸包；检查外向、无退化面、反向边配对、Euler 关系、面积和体积。另有 8000 组精确有理数方向参考，含共面、nextafter 近共面、不同指数和减法残差。原先放大整数点导致浮点行列式误判的反例保留。

模板题使用：P4724 三维凸包

题意：4≤n≤2000 个实数点，输出三维凸包表面积，保留三位小数。题面未给坐标界。驱动对纯平面输入采用正反双面包裹面积约定；核心 area() 仍返回平面单面面积。线上状态见验证记录。

所需模板：Hull3D（第 46 页，2.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4724

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<Hull3D<>::Point> p(n);
    for (auto &v : p) cin >> v.x >> v.y >> v.z;
    Hull3D hull(p);
    long double area = hull.area();
    if (hull.dimension == 2) area *= 2;
    cout << fixed << setprecision(3) << area << '\n';
}
```

2.3 整数凸多面体：体积重心与表面距离

代码依赖：Hull3D（第 46 页，2.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

先构造 Hull3D<long long> h，再构造 Hull3DMetrics m(h)，h 须存活且不修改。整数点和查询点坐标绝对值 $\le 10^9$。centroid() 返回 optional<array<long double,3» 体积重心；clearance() 返回重心到表面的最短距离，低维均为空。plane_distance(p,i) 为整数点到 faces[i] 所在无限平面的距离，不是到三角形的距离。planes() 给每个几何平面的代表三角面下标；二维为 1 个、点线为空。三角面数直接读 h.faces.size()。

构造 O(n+F)，重心 O(n)，距离 O(nF)，平面分组 $O(F^2)$，空间 O(n+F)。128 位精确累加非负四面体质量，距离通过非负仿射积分计算，避免薄凸包的重心点积抵消；最终浮点测量仍有舍入。mass、total 只供实现，不应改写。

字段与非负四面体质量：

```
// Integer coordinates |x|, |y|, |z| <= 1e9. Keep h alive and unchanged.
struct Hull3DMetrics
{
    using Hull = Hull3D<long long>;
    using Point = Hull::Point;
    const Hull &h;
    vector<__int128> mass;
    __int128 total = 0;

    explicit Hull3DMetrics(const Hull &h) : h(h), mass(h.p.size())
    {
        if (h.dimension != 3) return;
        for (auto f : h.faces)
        {
            __int128 v = Hull::orient(h.p[0], h.p[f[0]], h.p[f[1]], h.p[f[2]]);
            mass[0] += v;
            for (int u : f) mass[u] += v;
            total += 4 * v;
        }
    }
```

体积重心与整数点到平面距离（接同一结构体）：

```
    // Uniform volume centroid; absent for dimension < 3.
    optional<array<long double, 3>> centroid() const
    {
        if (total == 0) return nullopt;
        array<long long, 3> base{h.p[0].x, h.p[0].y, h.p[0].z};
        for (Point p : h.p)
        {
            base[0] = min(base[0], p.x);
            base[1] = min(base[1], p.y);
            base[2] = min(base[2], p.z);
        }
        array<long double, 3> result{};
        for (int i = 0; i < int(h.p.size()); i++)
        {
            array<long long, 3> p{h.p[i].x, h.p[i].y, h.p[i].z};
            for (int k = 0; k < 3; k++)
                result[k] += (long double)mass[i] * (p[k] - base[k]);
        }
        for (int k = 0; k < 3; k++) result[k] = base[k] + result[k] / total;
        return result;
    }
    long double normal_length(int face) const
    {
        auto f = h.faces[face];
        auto v = Hull::normal(h.p[f[0]], h.p[f[1]], h.p[f[2]]);
        return hypotl(hypotl((long double)v[0], (long double)v[1]),
                      (long double)v[2]);
    }
    // Distance to the infinite plane, NOT to the bounded triangle.
    long double plane_distance(Point p, int face) const
    {
        auto f = h.faces[face];
        auto v = Hull::orient(h.p[f[0]], h.p[f[1]], h.p[f[2]], p);
        return fabsl((long double)v) / normal_length(face);
    }
```

重心到表面距离及几何面分组（接同一结构体）：

```
    // Distance from the volume centroid to the surface; O(n * faces).
    optional<long double> clearance() const
    {
        if (total == 0) return nullopt;
        long double answer = numeric_limits<long double>::infinity();
        for (int i = 0; i < int(h.faces.size()); i++)
        {
            auto f = h.faces[i];
            long double sum = 0;
            for (int j = 0; j < int(h.p.size()); j++)
            {
                auto d = -Hull::orient(h.p[f[0]], h.p[f[1]], h.p[f[2]], h.p[j]);
                sum += (long double)mass[j] * (long double)d;
            }
            answer = min(answer, sum / total / normal_length(i));
        }
        return answer;
    }
    // One representative triangle per geometric plane; O(faces^2).
    vector<int> planes() const
    {
        vector<int> result;
        for (int i = 0; i < int(h.faces.size()); i++)
        {
            bool found = false;
            for (int j : result)
            {
                auto f = h.faces[j];
                bool same = true;
                for (int u : h.faces[i])
                    if (Hull::orient(h.p[f[0]], h.p[f[1]], h.p[f[2]], h.p[u]) != 0)
                        same = false;
                if (same)
                {
                    found = true;
                    break;
                }
            }
            if (!found) result.push_back(i);
        }
        return result;
    }
};
```

精度约定 取凸包点 p[0] 作公共四面体顶点，各外向面的六倍体积 v 非负，四顶点各加 v，total 为四倍六倍体积。坐标界 M 下 total 不超过 $192M^3$，128 位安全。各坐标先减最小值后非负求和；距离则对面行列式作非负质量积分，不用近似重心相减。浮点乘除和范数仍有舍入，重心接近零时不保证相对误差。

验证范围 509 组精确有理矩与支撑平面真值，六种构建通过；含行列式 1 的极薄整数四面体。原 kuangbin 完整 246 行与驱动、最小抄写版各 102 组 EOF 对拍。HDU4273 记录 41022769 已 AC，15MS、1844K；原题是整数坐标比赛应用，不据此声明任意实数测量已覆盖。

应用题补充：HDU4273 Rescue：重心到表面距离

题意：EOF 多组，4≤n≤100，整数坐标绝对值 ≤10000，输入均为非退化凸多面体顶点。输出三位小数。使用非负顶点质量直接积分各平面距离，避免薄凸包相减。

所需模板：Hull3D（第 46 页，2.2 节）、Hull3DMetrics（第 56 页，2.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://acm.hdu.edu.cn/showproblem.php?pid=4273

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    while (cin >> n)
    {
        vector<Hull3D<long long>::Point> p(n);
        for (auto &v : p) cin >> v.x >> v.y >> v.z;
        Hull3D<long long> h(p);
        Hull3DMetrics metrics(h);
        cout << fixed << setprecision(3) << *metrics.clearance() << '\n';
    }
}
```

接口用法演示：体积重心、代表面和原点到面平面的距离

演示约定：输入 n 及 n 个整数点。输出重心和表面距离（低维输出 NO VOLUME），再输出各几何平面的三角面编号、原输入顶点编号及原点到无限平面的距离。全部下标从 0 开始。

所需模板：Hull3D（第 46 页，2.2 节）、Hull3DMetrics（第 56 页，2.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://github.com/gsh20040816/CPCtemplate/blob/main/docs/HULL3D-METRICS.md

```
int main()
{
    int n;
    cin >> n;
    vector<Hull3D<long long>::Point> p(n);
    for (auto &v : p) cin >> v.x >> v.y >> v.z;
    Hull3D<long long> h(p);
    Hull3DMetrics metrics(h);
    cout << fixed << setprecision(12);
    auto center = metrics.centroid();
    if (!center)
        cout << "NO VOLUME\n";
    else
    {
        for (auto x : *center) cout << x << ' ';
        cout << *metrics.clearance() << '\n';
    }
    auto planes = metrics.planes();
    cout << planes.size() << '\n';
    for (int i : planes)
    {
        cout << i << ' ';
        for (int v : h.faces[i]) cout << h.id[v] << ' ';
        cout << metrics.plane_distance({0, 0, 0}, i) << '\n';
    }
}
```

2.4 实数三维向量与夹角

Point 为 long double 三坐标，支持加减、数乘/除和精确字典序；排序禁用 epsilon。dot、cross、norm 为点积、叉积、长度；unit 和 angle 要求非零向量，angle 返回 $0..\pi$ 的有向向量夹角，弧度制。unit(a)*r 变换长度；点间距离为 norm(a-b)。有限输入且中间量不溢出/下溢，极端尺度须预缩放；浮点不是精确共面判定。O(1) 时间空间。

```
struct RealSpace
{
    using R = long double;

    struct Point
    {
        R x = 0, y = 0, z = 0;

        Point operator+(Point b) const
        {
            return {x + b.x, y + b.y, z + b.z};
        }

        Point operator-(Point b) const
        {
            return {x - b.x, y - b.y, z - b.z};
        }

        Point operator*(R k) const
        {
            return {x * k, y * k, z * k};
        }

        Point operator/(R k) const
        {
            return {x / k, y / k, z / k};
        }

        bool operator<(Point b) const
        {
            return tie(x, y, z) < tie(b.x, b.y, b.z);
        }
    };
```

继续三维运算（接上页同一结构体）：

```
    static R dot(Point a, Point b)
    {
        return a.x * b.x + a.y * b.y + a.z * b.z;
    }

    static Point cross(Point a, Point b)
    {
        return {a.y * b.z - a.z * b.y,
                a.z * b.x - a.x * b.z,
                a.x * b.y - a.y * b.x};
    }

    static R norm(Point a)
    {
        return hypot(a.x, a.y, a.z);
    }

    static Point unit(Point a)
    {
        R len = norm(a);
        assert(len > 0);
        return a / len;
    }

    // Directed vectors: angle in [0, pi], both must be nonzero.
    static R angle(Point a, Point b)
    {
        a = unit(a);
        b = unit(b);
        return atan2(norm(cross(a, b)), dot(a, b));
    }
};
```

接口用法演示：三维向量运算（接口演示）

演示约定：EOF 输入两个向量 a,b，输出点积、叉积、两长度和 0..π 夹角；任一向量为零时夹角输出-1。不是正式 OJ 题。

所需模板：RealSpace（第 61 页，2.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：

```
int main()
{
    using G = RealSpace;
    G::Point a, b;
    cout << fixed << setprecision(15);
    while (cin >> a.x >> a.y >> a.z >> b.x >> b.y >> b.z)
    {
        auto c = RealSpace::cross(a, b);
        cout << G::dot(a, b) << '\n';
        cout << c.x << ' ' << c.y << ' ' << c.z << '\n';
        cout << G::norm(a) << ' ' << G::norm(b) << '\n';
        if (G::norm(a) > 0 && G::norm(b) > 0)
            cout << G::angle(a, b) << '\n';
        else
            cout << -1 << '\n';
    }
    return 0;
}
```

2.5 三维直线投影、线段距离与绕轴旋转

代码依赖：RealSpace（第 61 页，2.4 节）。抄写时一并准备；标准库类型不列入算法依赖。

Line3(p,d) 表示 p+t*d，注意第二参数是方向；through(a,b) 从两个端点构造。线段限制 0≤t≤1。projection/segment_projection 返回最近点，distance/segment_distance 返回距离，on_segment(q,eps) 使用绝对距离容差。d=0 时这些操作按点 p 处理；rotate(q,rad) 必须 d 非零，绕 p+t*d 按右手法则旋转，角度单位弧度。公开 p,d。依赖 RealSpace；O(1)。近退化浮点运算须按输入尺度分析误差。

```
struct Line3 : RealSpace
{
    Point p, d; // p + t*d; segment uses 0 <= t <= 1.

    Line3(Point p = {}, Point d = {}) : p(p), d(d)
    {
    }

    static Line3 through(Point a, Point b)
    {
        return Line3(a, b - a);
    }

    Point projection(Point q) const
    {
        if (norm(d) == 0) return p;
        Point u = unit(d);
        return p + u * dot(q - p, u);
    }

    Point segment_projection(Point q) const
    {
        R len = norm(d);
        if (len == 0) return p;
        Point u = d / len;
        R t = clamp(dot(q - p, u), R(0), len);
        return p + u * t;
    }
```

继续三维运算（接上页同一结构体）：

```
    R distance(Point q) const
    {
        if (norm(d) == 0) return norm(q - p);
        return norm(cross(unit(d), q - p));
    }

    R segment_distance(Point q) const
    {
        return norm(q - segment_projection(q));
    }

    bool on_segment(Point q, R eps = 1e-10L) const
    {
        assert(eps >= 0);
        return segment_distance(q) <= eps;
    }

    // Right-hand rotation about p + t*d; d must be nonzero.
    Point rotate(Point q, R rad) const
    {
        Point u = unit(d), v = q - p;
        R c = cos(rad), s = sin(rad);
        return p + v * c + cross(u, v) * s + u * (dot(u, v) * (1 - c));
    }
};
```

接口用法演示：三维直线/线段投影与绕轴旋转（接口演示）

演示约定：EOF 输入端点 a,b、查询点 q 和弧度 rad；输出直线/线段垂足、两距离、线段包含判定及绕 a 到 b 方向轴的旋转结果。a=b 时按点计算距离和投影，旋转输出 no axis。

所需模板：RealSpace（第 61 页，2.4 节）、Line3（第 64 页，2.5 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：

```
int main()
{
    using G = RealSpace;
    G::Point a, b, q;
    G::R rad;
    cout << fixed << setprecision(15);
    while (cin >> a.x >> a.y >> a.z >> b.x >> b.y >> b.z
               >> q.x >> q.y >> q.z >> rad)
    {
        auto l = Line3::through(a, b);
        auto p = l.projection(q);
        auto s = l.segment_projection(q);
        cout << p.x << ' ' << p.y << ' ' << p.z << '\n';
        cout << s.x << ' ' << s.y << ' ' << s.z << '\n';
        cout << l.distance(q) << ' ' << l.segment_distance(q) << '\n';
        cout << l.on_segment(q) << '\n';
        if (G::norm(l.d) > 0)
        {
            auto r = l.rotate(q, rad);
            cout << r.x << ' ' << r.y << ' ' << r.z << '\n';
        }
        else
            cout << "no axis\n";
    }
    return 0;
}
```

2.6 三维平面投影与分类求交

代码依赖：Line3（第 64 页，2.5 节）、RealSpace（第 61 页，2.4 节）。抄写时一并准备；标准库类型不列入算法依赖。

Plane3(p,normal) 要求非零法向量并单位化；through(a,b,c) 要求三点不共线；equation(n,h) 表示 n 点乘 x=h。signed_distance 保留法向符号，projection 返回垂足。intersect(line,q,eps,ang) 返回 1 单点、0 无交、-1 整条非退化线在面上，退化线按点处理。intersect(plane,line,eps,ang) 返回 1 交线、0 平行不同、-1 重合。仅返回 1 时写输出参数；交线 d 为单位向量。eps 为距离容差，ang 为单位方向的正弦容差；默认 1e-10 和 1e-12。近乎平行按容差分类，非精确拓扑判定；近乎平行求交误差会放大。公开 p 和单位 n，勿破坏单位不变量。依赖 RealSpace/Line3；O(1)。

```
struct Plane3 : RealSpace
{
    Point p, n; // n is a unit normal; dot(n, x - p) = 0.

    Plane3(Point p, Point normal) : p(p), n(unit(normal))
    {
    }

    static Plane3 through(Point a, Point b, Point c)
    {
        return Plane3(a, cross(b - a, c - a));
    }

    // dot(normal, x) = h; normal must be nonzero.
    static Plane3 equation(Point normal, R h)
    {
        Point u = unit(normal);
        return Plane3(u * (h / norm(normal)), u);
    }

    R signed_distance(Point q) const
    {
        return dot(n, q - p);
    }

    Point projection(Point q) const
    {
        return q - n * signed_distance(q);
    }
```

继续三维运算（接上页同一结构体）：

```
    // 1: one point; 0: none; -1: the nondegenerate line lies in plane.
    int intersect(Line3 l, Point &q, R eps = 1e-10L, R ang = 1e-12L) const
    {
        assert(eps >= 0 && 0 <= ang && ang < 1);
        R h = signed_distance(l.p);
        if (norm(l.d) == 0)
        {
            if (abs(h) > eps) return 0;
            q = l.p;
            return 1;
        }
        Point u = unit(l.d);
        R t = dot(n, u);
        if (abs(t) <= ang) return abs(h) <= eps ? -1 : 0;
        q = l.p - u * (h / t);
        return 1;
    }

    // 1: one line; 0: distinct parallel planes; -1: coincident planes.
    int intersect(Plane3 f, Line3 &l, R eps = 1e-10L, R ang = 1e-12L) const
    {
        assert(eps >= 0 && 0 <= ang && ang < 1);
        Point d = cross(n, f.n);
        R len = norm(d);
        if (len <= ang) return abs(signed_distance(f.p)) <= eps ? -1 : 0;
        d = d / len;
        Point v = cross(d, n);
        Point q = p + v * (dot(f.n, f.p - p) / dot(f.n, v));
        l = Line3(q, d);
        return 1;
    }
};
```

接口用法演示：平面投影和线面求交（接口演示）

演示约定：EOF 输入非零法向 n、常数 h、查询点 q 及直线端点 a,b，平面为 n 点乘 x=h。输出有符号距离、垂足和交点分类；仅 type=1 时输出单点。type=-1 为整条非退化线在面上，type=0 无交。

所需模板：RealSpace（第 61 页，2.4 节）、Line3（第 64 页，2.5 节）、Plane3（第 67 页，2.6 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：

```
int main()
{
    using G = RealSpace;
    G::Point n, q, a, b;
    G::R h;
    cout << fixed << setprecision(15);
    while (cin >> n.x >> n.y >> n.z >> h
               >> q.x >> q.y >> q.z
               >> a.x >> a.y >> a.z >> b.x >> b.y >> b.z)
    {
        auto f = Plane3::equation(n, h);
        auto p = f.projection(q);
        cout << f.signed_distance(q) << '\n';
        cout << p.x << ' ' << p.y << ' ' << p.z << '\n';
        G::Point x;
        int type = f.intersect(Line3::through(a, b), x);
        cout << type << '\n';
        if (type == 1)
            cout << x.x << ' ' << x.y << ' ' << x.z << '\n';
    }
    return 0;
}
```

接口用法演示：两平面夹角与交线（接口演示）

演示约定：EOF 输入两平面的非零法向 a,b 及常数 h,k（顺序 a,h,b,k），输出有向法向夹角及求交类型；type=1 时输出交线上一点和单位方向；-1 重合、0 平行不同。无向平面夹角可取 min(theta,π-theta)。

所需模板：RealSpace（第 61 页，2.4 节）、Line3（第 64 页，2.5 节）、Plane3（第 67 页，2.6 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：

```
int main()
{
    using G = RealSpace;
    G::Point a, b;
    G::R h, k;
    cout << fixed << setprecision(15);
    while (cin >> a.x >> a.y >> a.z >> h >> b.x >> b.y >> b.z >> k)
    {
        auto f = Plane3::equation(a, h);
        auto g = Plane3::equation(b, k);
        cout << G::angle(f.n, g.n) << '\n';
        Line3 l;
        int type = f.intersect(g, l);
        cout << type << '\n';
        if (type == 1)
        {
            cout << l.p.x << ' ' << l.p.y << ' ' << l.p.z << '\n';
            cout << l.d.x << ' ' << l.d.y << ' ' << l.d.z << '\n';
        }
    }
    return 0;
}
```

第三章　距离

3.1　点到闭线段的距离

代码依赖：RealPlane（第 1 页，1.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

segment_distance_real(p,a,b) 返回点 p 到闭线段 ab 的 long double 距离。a=b 时返回到端点的距离；否则把投影参数截断至 [0,1]。要求有限且尺度适当的输入；O(1) 时间和空间。

```
RealPlane::R
segment_distance_real(RealPlane::Point p, RealPlane::Point a, RealPlane::Point b)
{
    using G = RealPlane;
    using R = G::R;
    using Point = G::Point;
    Point v = b - a;
    R d = G::dot(v, v);
    if (d == 0) return G::norm(p - a);
    R t = clamp(G::dot(p - a, v) / d, R(0), R(1));
    return G::norm(p - (a + v * t));
}
```

模板题使用：AOJ CGL_2_D

题意：求两条闭线段间距离，q 不超过 1000，整数坐标绝对值不超过 10000。先用 IntegerPlane 精确判相交（含接触和共线重叠），相交输出 0；否则取四个端点到另一线段距离的最小值。单点距离接口参数为 (p,a,b)。题目线段非退化，点线段属于额外接口边界。

所需模板：IntegerPlane（第 14 页，1.8 节）、RealPlane（第 1 页，1.1 节）、segment_distance_real（第 71 页，3.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://onlinejudge.u-aizu.ac.jp/problems/CGL_2_D

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int q;
    cin >> q;
    cout << fixed << setprecision(12);
    while (q--)
    {
        vector<IntegerPlane::Point> p(4);
        vector<RealPlane::Point> a(4);
        for (int i = 0; i < 4; i++)
        {
            cin >> p[i].x >> p[i].y;
            a[i] = {(long double)p[i].x, (long double)p[i].y};
        }
        long double ans = 0;
        if (!IntegerPlane::intersect(p[0], p[1], p[2], p[3]))
            ans = min({segment_distance_real(a[0], a[2], a[3]),
                       segment_distance_real(a[1], a[2], a[3]),
                       segment_distance_real(a[2], a[0], a[1]),
                       segment_distance_real(a[3], a[0], a[1])});
        cout << ans << '\n';
    }
}
```

第四章　Pick 定理

4.1　Pick 定理：格点、剖分与孔洞

简单、非退化、顶点为整数点且无孔的多边形，令 A 为普通非负面积，I 为严格内部格点数，B 为边界上的不同格点数，则

$$A = I + \frac{B}{2} - 1, \qquad I = \frac{|S| - B + 2}{2}, \qquad S = \sum_i (x_i y_{i+1} - y_i x_{i+1}).$$

S 是有向二倍面积，先取绝对值再代入；不要对每一条叉积分别取绝对值。有向面积接口见第 1.11 节（第 19 页），组合用法见第 1.11 节（第 21 页）。

边界为什么求 gcd　一条非零整数线段的差向量为 (u, v)，令 $g = \gcd(|u|, |v|)$。

所有线段格点恰为起点加 $t(u/g, v/g)$，$t = 0, 1, \ldots, g$。这是因为 $(u/g, v/g)$ 互素，任何整数点沿这个方向的参数只能是整数。每条边只计起点、不计终点，就有 g 个格点；沿简单边界相加，每个顶点恰计一次，故

$$B = \sum_i \gcd(|x_{i+1} - x_i|, |y_{i+1} - y_i|).$$

共线的额外边界顶点不改变这个总和。重复走边或自交时不能沿用“每点恰计一次”的论证。

空格点三角形的面积　这里的“空”指除三个顶点外，内部和边上都没有格点。平移一个顶点到原点，取一条边 (u, v)；它必须是本原向量，即 $\gcd(|u|, |v|) = 1$。由裴蜀取 $ru + sv = 1$，整数矩阵

$$M = \begin{pmatrix} r & s \\ -v & u \end{pmatrix}, \qquad \det M = 1$$

将该边变成 $(1, 0)$，同时双向保留整数格点和面积。必要时把纵坐标取反，再作整数剪切 $x \leftarrow x - qy$，第三点可写成 (a, b)，$0 \le a < b$、$b > 0$。若 $b > 1$，高度 $y = 1$ 的闭截面为 $[a/b,\, 1 + (a-1)/b]$：$a = 0$ 时含格点 $(0, 1)$，$a \ge 1$ 时含 $(1, 1)$，都是非顶点格点，矛盾。所以 $b = 1$，面积恰为 $1/2$。仅三条边本原还不够：$(0, 0), (2, 1), (1, 2)$ 的边都本原，但内部有 $(1, 1)$。仅内部无格点也不够：$(0, 0), (k, 0), (0, 1)$ 的面积为 $k/2$，边上可能有其他格点。

相容剖分与 Euler 计数　先只使用多边形的整数顶点作三角剖分，不引入非整数辅助点，再不断用内部或边上的非顶点格点细分。点在公共边上时，必须同时细分两侧三角形，不能留下悬挂节点。每次细分都增加正面积三角形数，而每个三角形的二倍面积是正整数，数量被总二倍面积界住，因此过程终止。最终每个小三角形面积 $1/2$，所有格点都是剖分顶点，每条边界小边是本原线段。令 V, E, F 分别为顶点、边、区域内小三角形数（不计无界面），则

$$V = I + B, \qquad F = 2A, \qquad 3F = 2E - B, \qquad V - E + F = 1.$$

消去 V, E, F 即得 Pick 公式。这是定理证明；比赛计算不需要实际构造剖分。

孔洞、方向与不能套用的情况 若区域连通，外边界与 h 条孔洞边界都是简单整数多边形，被挖去的孔洞区域严格在外边界内且彼此不交、不接触、不嵌套，同样的相容剖分满足 $V-E+F=1-h$，所以

$$A = I + \frac{B}{2} - 1 + h, \qquad I = \frac{2A - B + 2}{2} - h.$$

I 不含孔洞内部及任意边界；B 包含外环与所有内环的边界点。面积为外环面积减各孔洞面积，或约定外环逆时针、内环顺时针后累加有向面积。例如外方形 $[0,4]^2$ 去掉孔洞 $(1,3)^2$，有 $A=12, B=24, I=0$；漏掉 $h=1$ 会错得 $I=1$。这个公式不是本单环用法额外实现了孔洞 API 的声明。

非整数顶点、自交、边界相接或面积退化均不在上述合同内。蝴蝶结 $(0,0),(2,2),(0,2),(2,0)$ 的有向面积为 0，却不能按 Pick 计数。检查结果非负或分子为偶数，也不能代替简单性检查；正式题是直接保证简单多边形的。

整数位宽与组合用法 现有整数几何合同为 $n \le 10^5$、$|x|, |y| \le C = 10^{12}$。差值和其绝对值可用 long long，但逐项叉积、总二倍面积以及最终分子使用 signed128。粗界 $|S| \le 2nC^2$、$B \le 2nC$，在该范围内都安全；内部点数可超过 long long，示例直接打印 128 位整数。CSES2193 的正式限制更窄：$3 \le n \le 10^5$、坐标绝对值不超过 10^9，只输入一个简单边界环。复用 `polygon_area2` 与标准库 gcd 即可，不新增一份鞋带公式算法。

参考：https://oi-wiki.org/geometry/pick/；https://cses.fi/problemset/task/2193。

第五章　凸包

5.1　凸多边形外点的两条切线

代码依赖：IntegerPlane（第 14 页，1.8 节）。抄写时一并准备；标准库类型不列入算法依赖。

p 须为严格凸、逆时针的整数多边形，无重复末点或共线边界顶点；3≤n≤INT_MAX/2，起始下标任意。所有顶点及 q 坐标绝对值 ≤10^12。返回 optional<pair<int,int» 的原 0-based 下标 (right,left)：对任意顶点 v，cross(q,p[right],v)≤0，cross(q,p[left],v)≥0。若切线贴整条边，选离 q 最近的端点。q 在内部或边界时返回 nullopt。O(log n) 时间、O(1) 额外空间，无预处理且输入不变；先扇形定位异态边，再二分可见边链两端。凸性、方向及数值范围均为前置条件，assert 不验证整个多边形；n<3 不在接口范围。散点先构造 integer_hull，若需原编号须另留映射。

```
optional<pair<int, int>> convex_tangents_i64(const vector<IntegerPlane::Point> &p,
                                             IntegerPlane::Point q)
{
    using G = IntegerPlane;
    assert(p.size() >= 3 && p.size() <= INT_MAX / 2);
    int n = p.size();
    auto visible = [&](int i)
    {
        return G::cross(p[i], p[(i + 1) % n], q) < 0;
    };
    auto sector = [&](int dir)
    {
        int l = 1, r = n - 1;
        while (l + 1 < r)
        {
            int m = (l + r) / 2;
            if (dir * G::cross(p[0], p[m], q) >= 0)
                l = m;
            else
                r = m;
        }
        return l;
    };
```

选择异态种子边并二分可见边链（接上页同一函数）：

```
    int a = G::sign(G::cross(p[0], p[1], q));
    int b = G::sign(G::cross(p[0], p[n - 1], q));
    int v, h;
    if (a < 0)
    {
        v = 0;
        h = b <= 0 ? n - 1 : sector(-1);
    }
    else if (b > 0)
    {
        v = n - 1;
        h = 0;
    }
    else
    {
        v = sector(1);
        h = 0;
        if (!visible(v)) return nullopt;
    }
    auto boundary = [&](int start, int end)
    {
        int l = 0, r = (end - start + n) % n;
        bool first = visible(start);
        while (l + 1 < r)
        {
            int m = (l + r) / 2;
            if (visible((start + m) % n) == first)
                l = m;
            else
                r = m;
        }
        return (start + r) % n;
    };
    return pair{boundary(h, v), boundary(v, h)};
}
```

接口用法演示：凸多边形外点切线（接口演示）

演示约定：自定义有效输入协议，非 OJ 题：输入 n m，3≤n≤100000、0≤m≤100000；随后 n 个严格凸 CCW 顶点，再 m 个查询点，坐标绝对值均 ≤10^12。每个外点输出原 0-based 编号 right left，整边相切选最近端点；内部或边界输出-1 -1。程序不验证凸性或方向，不接受重复末点；可从任意顶点开始列举。仅演示切线接口，不包含成都 2025 I 的旋转双指针计数。

所需模板：IntegerPlane（第 14 页，1.8 节）、convex_tangents_i64（第 75 页，5.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://github.com/gsh20040816/CPCtemplate/blob/main/src/compact/convex_tangents_i64.hpp

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<IntegerPlane::Point> p(n);
    for (auto &v : p) cin >> v.x >> v.y;
    while (m--)
    {
        IntegerPlane::Point q;
        cin >> q.x >> q.y;
        auto ans = convex_tangents_i64(p, q);
        if (ans)
            cout << ans->first << ' ' << ans->second << '\n';
        else
            cout << "-1 -1\n";
    }
}
```

应用题补充：ICPC Chengdu 2025 I / Inside Triangle

题意：多组严格嵌套的 CCW 凸多边形 P、Q，分别 3..300000 顶点，坐标绝对值 ≤10^9；T≤1000 且两种顶点总数分别 ≤500000。选 P 的 3 个不同顶点，使三角形非严格包含 Q，贴边也计数。输入 T，各组 n、n 个 P 点、m、m 个 Q 点。复用左切线、合法弦端点的单调性和前缀和，累计每个起点贡献后除 3；实际 O(n log m+n+m)，不是官方线性旋转切线版本。

所需模板：IntegerPlane（第 14 页，1.8 节）、convex_tangents_i64（第 75 页，5.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://codeforces.com/gym/106161/problem/I

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    using G = IntegerPlane;
    int t;
    cin >> t;
    while (t--)
    {
        int n, m;
        cin >> n;
        vector<G::Point> p(n);
        for (auto &v : p) cin >> v.x >> v.y;
        cin >> m;
        vector<G::Point> q(m);
        for (auto &v : q) cin >> v.x >> v.y;
        vector<int> f(2 * n);
        for (int i = 0, j = 1; i < n; i++)
        {
            int k = convex_tangents_i64(q, p[i])->second;
            j = max(j, i + 1);
            while (j + 1 < i + n && G::cross(p[i], p[(j + 1) % n], q[k]) >= 0)
                j++;
            f[i] = j;
            f[i + n] = j + n;
        }
        vector<long long> sum(2 * n + 1);
        for (int i = 0; i < 2 * n; i++) sum[i + 1] = sum[i] + f[i];
        long long ans = 0;
        for (int a = 0, g = 0, h = 0; a < n; a++)
        {
            while (f[g] < a + n) g++;
            h = max(h, a + 1);
            while (h <= f[a] && f[h] < g) h++;
            if (h <= f[a])
                ans += sum[f[a] + 1] - sum[h] - 1LL * (f[a] - h + 1) * (g - 1);
        }
        cout << ans / 3 << '\n';
    }
}
```

5.2 整数凸包：单调链

代码依赖：IntegerPlane（第 14 页，1.8 节）。抄写时一并准备；标准库类型不列入算法依赖。

integer_hull(points) 按值接收 IntegerPlane::Point 序列，输出严格逆时针凸包，不重复首点、不保留共线边内点。自动排序去重；全共线时保留两个端点，空集及单点直接返回。坐标范围同 IntegerPlane。O(n log n) 时间、O(n) 空间；供凸包包含、直径与 Minkowski 和调用。

```
vector<IntegerPlane::Point> integer_hull(vector<IntegerPlane::Point> p)
{
    using G = IntegerPlane;
    using Point = G::Point;
    sort(p.begin(), p.end());
    p.erase(unique(p.begin(), p.end()), p.end());
    int n = (int)p.size();
    if (n < 3) return p;
    vector<Point> h;
    for (Point v : p)
    {
        while (h.size() > 1 && G::cross(h[h.size() - 2], h.back(), v) <= 0)
            h.pop_back();
        h.push_back(v);
    }
    int low = (int)h.size();
    for (int i = n - 2; i >= 0; i--)
    {
        while ((int)h.size() > low && G::cross(h[h.size() - 2], h.back(), p[i]) <= 0)
            h.pop_back();
        h.push_back(p[i]);
    }
    h.pop_back();
    return h; // CCW, no repeated endpoint, no collinear interior points.
}
```

模板题使用：Library Checker static_convex_hull

题意：多组求整数点集的凸包，去重并去掉边上非顶点。输出顶点数及逆时针顶点，不重复首点；空集输出 0，共线时仅保留两端。传 move 交出点集，避免按值传参时额外复制。

所需模板：IntegerPlane（第 14 页，1.8 节）、integer_hull（第 79 页，5.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/static_convex_hull

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int tests;
    cin >> tests;
    while (tests--)
    {
        int n;
        cin >> n;
        vector<IntegerPlane::Point> points(n);
        for (auto &p : points) cin >> p.x >> p.y;
        auto hull = integer_hull(move(points));
        cout << hull.size() << '\n';
        for (auto p : hull) cout << p.x << ' ' << p.y << '\n';
    }
}
```

应用题补充：Luogu P2742

题意：求围住所有点的最短围栏长度。坐标绝对值 $\leq 10^6$、最多两位小数；按字符串读成放大 100 倍的整数，精确求凸包，最后才开方并除以 100。全共线时周长为两端距离的两倍，全部重合为 0。USACO 来源作为应用补充。

所需模板：IntegerPlane（第 14 页，1.8 节）、integer_hull（第 79 页，5.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P2742

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    auto read = []()
    {
        string s;
        cin >> s;
        bool negative = s[0] == '-';
        int i = negative || s[0] == '+';
        long long value = 0;
        while (i < (int)s.size() && s[i] != '.') value = value * 10 + s[i++] - '0';
        i++;
        for (int j = 0; j < 2; j++)
        {
            value *= 10;
            if (i < (int)s.size()) value += s[i++] - '0';
        }
        return negative ? -value : value;
    };
    int n;
    cin >> n;
    vector<IntegerPlane::Point> p(n);
    for (auto &v : p)
    {
        v.x = read();
        v.y = read();
    }
    auto h = integer_hull(move(p));
    long double length = 0;
    for (int i = 0; i < (int)h.size(); i++)
        length += sqrtl((long double)IntegerPlane::dist2(h[i], h[(i + 1) % h.size()]));
    cout << fixed << setprecision(2) << length / 100 << '\n';
}
```

5.3 点在整数凸包内（二分）

代码依赖：IntegerPlane（第 14 页，1.8 节）、polygon_contains（第 22 页，1.12 节）。抄写时一并准备；标准库类型不列入算法依赖。

convex_contains_i64(p,q) 返回 0 外部、1 边界、2 内部。要求严格逆时针凸包，无共线边内点、不重复首点，例如 integer_hull 的输出；允许循环旋转起点。空集、单点、线段转用 polygon_contains。坐标范围同 IntegerPlane，O(log n) 时间、O(1) 空间。

```
int convex_contains_i64(const vector<IntegerPlane::Point> &p, IntegerPlane::Point q)
{
    using G = IntegerPlane;
    using I = G::I;
    int n = (int)p.size();
    if (n < 3) return polygon_contains(p, q);
    I a = G::cross(p[0], p[1], q), b = G::cross(p[0], p[n - 1], q);
    if (a < 0 || b > 0) return 0;
    if (!a) return G::on_segment(q, p[0], p[1]) ? 1 : 0;
    if (!b) return G::on_segment(q, p[0], p[n - 1]) ? 1 : 0;
    int l = 1, r = n - 1;
    while (l + 1 < r)
    {
        int m = (l + r) / 2;
        if (G::cross(p[0], p[m], q) >= 0)
            l = m;
        else
            r = m;
    }
    I c = G::cross(p[l], p[r], q);
    return c < 0 ? 0 : c == 0 ? 1 : 2;
}
```

联合使用示例见第 84 页（Luogu P4557 [JSOI2018] 战争），其中直接调用本模板。

相关应用：固定导航没有独立 Minkowski 和条目，按凸包的相关应用归档。

5.4 凸多边形 Minkowski 和

代码依赖：IntegerPlane（第 14 页，1.8 节）、integer_hull（第 79 页，5.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

minkowski_sum(a,b) 接受不重复首点的严格逆时针凸包；空输入返回空集，点和线段允许作为退化凸包。

点类型 IntegerPlane::Point；输入坐标及两集合坐标和的绝对值不超过 10^{12}。结果不重复首点，非退化时沿逆时针边方向合并，平行同向边同时推进；O(n+m) 时间。

退化输入枚举与另一凸包各点的和再求凸包，O((n+m) log(n+m))。按值传入，内部旋转不修改调用方；依赖 IntegerPlane 的 integer_hull/cross。

```
vector<IntegerPlane::Point> minkowski_sum(vector<IntegerPlane::Point> a,
                                          vector<IntegerPlane::Point> b)
{
    using G = IntegerPlane;
    using P = G::Point;
    using I = G::I;
    if (a.empty() || b.empty()) return {};
    auto plus = [](P u, P v)
    {
        return P{u.x + v.x, u.y + v.y};
    };
    if (a.size() < 3 || b.size() < 3)
    {
        vector<P> p;
        for (P u : a)
            for (P v : b) p.push_back(plus(u, v));
        return integer_hull(p);
    }
    auto start = [](vector<P> &p)
    {
        auto it = min_element(
            p.begin(), p.end(), [](P u, P v) { return tie(u.y, u.x) < tie(v.y, v.x); });
        rotate(p.begin(), it, p.end());
    };
    start(a);
    start(b);
    int n = a.size(), m = b.size(), i = 0, j = 0;
    vector<P> result;
```

合并凸包边（接上页同一函数）：

```
    while (i < n || j < m)
    {
        result.push_back(plus(a[i % n], b[j % m]));
        if (i == n)
        {
            ++j;
            continue;
        }
        if (j == m)
        {
            ++i;
            continue;
        }
        P u{a[(i + 1) % n].x - a[i].x, a[(i + 1) % n].y - a[i].y};
        P v{b[(j + 1) % m].x - b[j].x, b[(j + 1) % m].y - b[j].y};
        I c = G::cross({0, 0}, u, v);
        if (c >= 0) ++i;
        if (c <= 0) ++j;
    }
    return result;
}
```

应用题补充：Luogu P4557 [JSOI2018] 战争

题意：两组点各自凸包为领地；每次将第二组平移 d，问两领地是否相交，接触边界也输出 1。先构造 A+(-B)，再判断 d 是否在其闭包内。n、m、q 各至 10 万，坐标和平移量绝对值至 10^8。比赛应用，不计正式模板题覆盖。

所需模板：IntegerPlane（第 14 页，1.8 节）、integer_hull（第 79 页，5.2 节）、minkowski_sum（第 83 页，5.4 节）、polygon_contains（第 22 页，1.12 节）、convex_contains_i64（第 82 页，5.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4557

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, q;
    cin >> n >> m >> q;
    vector<IntegerPlane::Point> a(n), b(m);
    for (auto &v : a) cin >> v.x >> v.y;
    for (auto &v : b)
    {
        cin >> v.x >> v.y;
        v.x = -v.x;
        v.y = -v.y;
    }
    auto p = minkowski_sum(integer_hull(a), integer_hull(b));
    while (q--)
    {
        IntegerPlane::Point v;
        cin >> v.x >> v.y;
        cout << (convex_contains_i64(p, v) != 0) << '\n';
    }
}
```

5.5 支撑点查询恢复下凸包

静态 build(left,right,oracle) 返回从左到右的严格下凸包顶点，Point 为 {x,y}。有限非空整数点集无需显式给出；回调 oracle(a,b) 必须精确返回使 a*x+b*y 最大的可行点，允许任意最优解。left/right 必须分别为最小/最大横坐标中纵坐标最低的点；全体同横坐标时二者必须相同。所有坐标绝对值不超过 10^18，方向差使用 long long，叉积先提升至 int128；回调内部也须保证数值精确且无溢出。对弦 a→b 查询方向 (b.y-a.y,a.x-b.x)，严格位于弦下方则递归分治，否则确认边；输出时删除共线中间点。若分治找到 K 个严格改善支撑点，查询次数为 2K+1（同横坐标时为 0），额外时间、空间和最坏递归深度为 O(K+1)。K 包括可能返回的共线非顶点，不将其等同于最终顶点数。不能直接传浮点近似优化器。二分图闭包组合：点为 (|S|,|N(S)|)，左点赋权 a、右点赋权 b，左 → 邻点依赖；下凸包方向满足 a≥0,b<0，最优闭包的右点恰为邻集。端点为 (0,0) 和 (左点数, 全部左点的邻点数)，有孤立右点时后者不等于全部右点数。该组合已做子集枚举对拍，尚不代表关联赛题整题通过。

```
struct SupportHull
{
    using ll = long long;
    using I = __int128_t;

    struct Point
    {
        ll x, y;
    };

    static I cross(Point a, Point b, Point c)
    {
        return (I(b.x) - a.x) * (I(c.y) - a.y) - (I(b.y) - a.y) * (I(c.x) - a.x);
    }
```

```
    // oracle(a,b) maximizes a*x+b*y exactly; |coordinates| <= 1e18.
    // left/right: lowest points at the minimum/maximum x.
    template <class Oracle>
    static vector<Point> build(Point left, Point right, Oracle &&oracle)
    {
        assert(left.x <= right.x);
        vector<Point> hull{left};
        auto dfs = [&](auto &&self, Point a, Point b) -> void
        {
            Point c = oracle(b.y - a.y, a.x - b.x);
            I turn = cross(a, b, c);
            assert(turn <= 0);
            if (turn < 0)
            {
                assert(a.x < c.x && c.x < b.x);
                self(self, a, c);
                self(self, c, b);
            }
            else
            {
                while (hull.size() >= 2 &&
                       cross(hull[hull.size() - 2], hull.back(), b) == 0)
                    hull.pop_back();
                hull.push_back(b);
            }
        };
        if (left.x < right.x)
            dfs(dfs, left, right);
        else
            assert(left.y == right.y);
        return hull;
    }
};
```

接口用法演示：自定义静态下凸链支撑查询

演示约定：自定义协议，非 OJ 题：0≤n≤2000，0≤q≤200000；点和方向坐标绝对值 ≤10^18，查询 b≤0。先输出严格下凸链长度及坐标，再输出 max(a*x+b*y) 及字典序最小的最大点；空集每次输出 EMPTY。用最左/最右横坐标上的最低点和全点集精确扫描 oracle 调用 build，再二分首个最大顶点；同 x 只留最低 y，不承诺原下标或向上查询。扫描回调使构造 O(n^2)、存储/递归 O(n)，查询 O(q log(max(2,n)))；显式点云通常用普通单调链更合适，本例演示隐式点集的 oracle 适配。生成方向 ≤2×10^18，回调点积 ≤4×10^36、叉积 ≤8×10^36，以 128 位精确运算；不扩展到完整 signed64 域。

所需模板：SupportHull（第 85 页，5.5 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://github.com/gsh20040816/CPCtemplate/blob/main/src/compact/support_hull.hpp

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    using P = SupportHull::Point;
    using I = __int128_t;
    int n, q;
    cin >> n >> q;
    vector<P> points(n), hull;
    for (auto &p : points) cin >> p.x >> p.y;
    auto less = [](P p, P r) { return p.x != r.x ? p.x < r.x : p.y < r.y; };
    auto dot = [](P p, long long a, long long b) { return I(a) * p.x + I(b) * p.y; };
    if (n)
    {
        P left = *min_element(points.begin(), points.end(), less), right = left;
        for (P p : points)
            if (p.x > right.x || (p.x == right.x && p.y < right.y)) right = p;
        hull = SupportHull::build(left, right, [&](long long a, long long b)
        {
            P best = points[0];
            for (P p : points)
                if (dot(p, a, b) > dot(best, a, b) ||
                    (dot(p, a, b) == dot(best, a, b) && less(p, best))) best = p;
            return best;
        });
    }
    cout << hull.size() << '\n';
    for (P p : hull) cout << p.x << ' ' << p.y << '\n';
```

继续处理操作（接上页同一 main）：

```
    while (q--)
    {
        long long a, b;
        cin >> a >> b; // Custom contract: b <= 0; upward support is excluded.
        if (hull.empty()) { cout << "EMPTY\n"; continue; }
        int l = 0, r = int(hull.size()) - 1;
        while (l < r)
        {
            int m = (l + r) / 2;
            if (dot(hull[m], a, b) < dot(hull[m + 1], a, b)) l = m + 1;
            else r = m; // First maximizing vertex, including edge/zero ties.
        }
        I value = dot(hull[l], a, b);
        if (value < 0) { cout << '-'; value = -value; }
        string digits;
        do { digits += char('0' + value % 10); value /= 10; } while (value);
        reverse(digits.begin(), digits.end());
        cout << digits << ' ' << hull[l].x << ' ' << hull[l].y << '\n';
    }
}
```

第六章　扫描线

6.1　整数轴对齐矩形面积并

调用 rectangle_union_area(rect)，每项按左、下、右、上存储四个 long long。端点有序，各坐标绝对值不超过 10^{18}；零面积矩形忽略，空集返回 0。返回精确 ___int128 面积；边界接触不增加面积，输入不变，多次调用独立。扫描线处理进出事件，纵向叶子代表相邻坐标间的半开区间。cover 仅记录整段覆盖次数，不下传；非零时取整段长度，为零时合并孩子。同横坐标事件间宽度为 0。时间 O(n log(n+1))、空间 O(n)，递归深度 O(log(n+1))；矩形数至多 268435454，且内存可用。题目坐标在 0 到 10^9，故驱动可将结果转 long long；一般输入不要直接窄化。不含周长、浮点或动态更新。

```
__int128 rectangle_union_area(const vector<array<long long, 4>> &rect)
{
    using ll = long long;
    assert(rect.size() <= (INT_MAX - 8) / 8);
    vector<array<ll, 4>> events;
    vector<ll> y;
    for (auto [l, d, r, u] : rect)
    {
        for (ll x : {l, d, r, u})
            assert(-1000000000000000000LL <= x && x <= 1000000000000000000LL);
        assert(l <= r && d <= u);
        if (l == r || d == u) continue;
        events.push_back({l, d, u, 1});
        events.push_back({r, d, u, -1});
        y.push_back(d);
        y.push_back(u);
    }
    if (events.empty()) return 0;
    sort(events.begin(), events.end());
    sort(y.begin(), y.end());
    y.erase(unique(y.begin(), y.end()), y.end());
    int n = int(y.size()) - 1;
    vector<int> cover(4 * n + 4);
    vector<ll> len(4 * n + 4);
```

覆盖长度维护与横向面积累加（接上页同一函数）：

```
    auto add = [&](auto &&self, int p, int l, int r, int a, int b, int v) -> void
    {
        if (a <= l && r <= b) cover[p] += v;
        else
        {
            int m = (l + r) / 2;
            if (a < m) self(self, p * 2, l, m, a, b, v);
            if (b > m) self(self, p * 2 + 1, m, r, a, b, v);
        }
        if (cover[p]) len[p] = y[r] - y[l];
        else len[p] = r - l == 1 ? 0 : len[p * 2] + len[p * 2 + 1];
    };
    __int128 area = 0;
    ll previous = events.front()[0];
    for (auto [x, d, u, v] : events)
    {
        area += (__int128)(x - previous) * len[1];
        int l = lower_bound(y.begin(), y.end(), d) - y.begin();
        int r = lower_bound(y.begin(), y.end(), u) - y.begin();
        add(add, 1, 0, n, l, r, int(v));
        previous = x;
    }
    return area;
}
```

模板题使用：Library Checker Area of Union of Rectangles

题意：输入 N 个轴对齐矩形，每行依次为 l,d,r,u。1≤N≤500000，0≤l<r≤10^9，0≤d<u≤10^9。直接按左、下、右、上存储，计算并集面积；不是面积和。所有矩形都在边长 10^9 的正方形内，最终面积 ≤10^18，可把 __int128 结果转成 long long 输出。

所需模板：rectangle_union_area（第 89 页，6.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/area_of_union_of_rectangles

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<array<long long, 4>> rect(n);
    for (auto &a : rect) for (auto &x : a) cin >> x;
    // The statement bounds the whole union by [0,1e9]^2.
    cout << (long long)rectangle_union_area(rect) << '\n';
}
```

第七章　旋转卡壳

7.1　整数凸包直径：旋转卡壳

代码依赖：IntegerPlane（第 14 页，1.8 节）。抄写时一并准备；标准库类型不列入算法依赖。

convex_diameter2(p,endpoints=nullptr) 返回最远两点距离平方，类型 __int128_t；不足两点返回 0。输入须为严格逆时针凸包，不重复首点，例如 integer_hull 的输出。坐标范围同 IntegerPlane；旋转卡壳 O(n) 时间、O(1) 额外空间。可传 pair<int,int> 的地址，接收凸包内的 0-based 端点下标；空集为 {-1,-1}，单点为 {0,0}，至少两点时端点不同。不保证并列最优端点的顺序。映射到原始输入编号由调用者完成。

```
IntegerPlane::I convex_diameter2(const vector<IntegerPlane::Point> &p,
                                 pair<int, int> *endpoints = nullptr)
{
    using G = IntegerPlane;
    using I = G::I;
    int n = (int)p.size();
    if (endpoints) *endpoints = n ? pair{0, n - 1} : pair{-1, -1};
    if (n < 2) return 0;
    if (n == 2) return G::dist2(p[0], p[1]);
    I ans = 0;
    auto update = [&](int a, int b)
    {
        I value = G::dist2(p[a], p[b]);
        if (value > ans)
        {
            ans = value;
            if (endpoints) *endpoints = {a, b};
        }
    };
    int j = 1;
    for (int i = 0; i < n; i++)
    {
        int k = (i + 1) % n;
        while (G::cross(p[i], p[k], p[(j + 1) % n]) > G::cross(p[i], p[k], p[j]))
            j = (j + 1) % n;
        update(i, j);
        update(k, j);
    }
    return ans;
}
```

模板题使用：Library Checker furthest_pair

题意：多组求距离最远的两个不同输入点编号。先建凸包，端点接口返回凸包下标，再按坐标映射回输入下标。题目保证至少两点；若凸包只剩一个点，所有坐标相同，输出 0、1。

所需模板：IntegerPlane（第 14 页，1.8 节）、integer_hull（第 79 页，5.2 节）、convex_diameter2（第 91 页，7.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/furthest_pair

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int tests;
    cin >> tests;
    while (tests--)
    {
        int n;
        cin >> n;
        vector<IntegerPlane::Point> points(n);
        for (auto &p : points) cin >> p.x >> p.y;
        auto hull = integer_hull(points);
        if (hull.size() == 1)
        {
            cout << "0 1\n";
            continue;
        }
        pair<int, int> ends;
        convex_diameter2(hull, &ends);
        int a = -1, b = -1;
        for (int i = 0; i < n; i++)
        {
            if (points[i] == hull[ends.first]) a = i;
            if (points[i] == hull[ends.second]) b = i;
        }
        cout << a << ' ' << b << '\n';
    }
}
```

应用题补充：Luogu P1452

题意：求最多 5 万个点的最远距离平方，坐标绝对值 $\le 10^4$。先求严格凸包再旋转卡壳；本题答案 $\le 8\times 10^8$，可转 long long 输出，不能照搬到通用万亿坐标。USACO 来源作为应用补充；返回端点的正式示例见 Library Checker furthest_pair。

所需模板：IntegerPlane（第 14 页，1.8 节）、integer_hull（第 79 页，5.2 节）、convex_diameter2（第 91 页，7.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P1452

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<IntegerPlane::Point> p(n);
    for (auto &v : p) cin >> v.x >> v.y;
    auto h = integer_hull(move(p));
    cout << (long long)convex_diameter2(h) << '\n';
}
```

第八章　半平面交

8.1　整数半平面的有界多边形交

bounded_polygon(lines) 求 a*x+b*y≤c 的交集闭包顶点，Line={a,b,c}，三个整数系数绝对值均 $\leq 10^9$。要求交集有界，空集也允许；只返回正面积多边形，点或线段交集返回空向量。无界输入不在接口范围内。Point={x,y,d} 表示 (x/d,y/d)，d>0，分数不必约分；按逆时针返回，不重复首点，相邻重复点已去除。排序与所有构造、判定使用整数和 int128，不使用 epsilon。O(n log n) 时间、O(n) 空间。浮点转换与面积计算需单独处理。

```
struct IntegerHalfplanes
{
    using I = __int128_t;
    using ll = long long;

    struct Line
    {
        ll a, b, c;
    };

    struct Point
    {
        ll x, y, d;
    };

    static I cross(Line u, Line v) { return I(u.a) * v.b - I(u.b) * v.a; }

    static int half(Line u) { return u.b < 0 || (u.b == 0 && u.a < 0); }

    static Point meet(Line u, Line v)
    {
        I d = cross(u, v);
        assert(d != 0);
        I x = I(u.c) * v.b - I(u.b) * v.c;
        I y = I(u.a) * v.c - I(u.c) * v.a;
        if (d < 0)
        {
            d = -d;
            x = -x;
            y = -y;
        }
        return {(ll)x, (ll)y, (ll)d};
    }
```

接上页同一结构体与函数:

```
35      static I value(Line u, Point p)
36      {
37          return I(u.a) * p.x + I(u.b) * p.y - I(u.c) * p.d;
38      }
39
40      static bool same(Point p, Point q)
41      {
42          return I(p.x) * q.d == I(q.x) * p.d && I(p.y) * q.d == I(q.y) * p.d;
43      }
44
45      static vector<Point> bounded_polygon(vector<Line> lines)
46      {
47          vector<Line> h;
48          for (auto u : lines)
49          {
50              assert(-1000000000 <= u.a && u.a <= 1000000000);
51              assert(-1000000000 <= u.b && u.b <= 1000000000);
52              assert(-1000000000 <= u.c && u.c <= 1000000000);
53              if (!u.a && !u.b)
54              {
55                  if (u.c < 0) return {};
56              }
57              else
58                  h.push_back(u);
59          }
60          sort(h.begin(),
61               h.end(),
62               [](Line u, Line v)
63               {
64                   if (half(u) != half(v)) return half(u) < half(v);
65                   I d = cross(u, v);
66                   if (d) return d > 0;
67                   ll x = u.a ? (u.a > 0 ? u.a : -u.a) : (u.b > 0 ? u.b : -u.b);
68                   ll y = v.a ? (v.a > 0 ? v.a : -v.a) : (v.b > 0 ? v.b : -v.b);
69                   return I(u.c) * y < I(v.c) * x;
70               });
```

接上页同一结构体与函数:

```
        deque<Line> q;
        for (int i = 0; i < (int)h.size(); i++)
        {
            Line u = h[i];
            if (i && half(u) == half(h[i - 1]) && cross(u, h[i - 1]) == 0) continue;
            while (q.size() >= 2 && value(u, meet(q[q.size() - 2], q.back())) > 0)
                q.pop_back();
            while (q.size() >= 2 && value(u, meet(q[0], q[1])) > 0) q.pop_front();
            if (!q.empty() && cross(q.back(), u) == 0) return {};
            q.push_back(u);
        }
        while (q.size() >= 3 && value(q.front(), meet(q[q.size() - 2], q.back())) > 0)
            q.pop_back();
        while (q.size() >= 3 && value(q.back(), meet(q[0], q[1])) > 0) q.pop_front();
        if (q.size() < 3 || cross(q.front(), q.back()) == 0) return {};
        vector<Point> answer;
        for (int i = 0; i < (int)q.size(); i++)
        {
            Point p = meet(q[i], q[(i + 1) % q.size()]);
            if (answer.empty() || !same(answer.back(), p)) answer.push_back(p);
        }
        if (answer.size() > 1 && same(answer.front(), answer.back())) answer.pop_back();
        if (answer.size() < 3) return {};
        return answer;
    }
};
```

应用题补充：Luogu P4196

题意：逆时针输入 2..10 个凸多边形，每个 3..50 个顶点、整数坐标绝对值 ≤1000，求交面积。每条有向边的左侧转为 a*x+b*y≤c，系数满足模板范围；交集必有界，空集或退化交集面积为 0。交点与面积各项分子分母先用整数，最后转浮点累加；CQOI 来源作为应用补充。

所需模板：IntegerHalfplanes（第 94 页，8.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4196

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    using H = IntegerHalfplanes;
    int n;
    cin >> n;
    vector<H::Line> lines;
    while (n--)
    {
        int m;
        cin >> m;
        vector<pair<long long, long long>> p(m);
        for (auto &[x, y] : p) cin >> x >> y;
        for (int i = 0; i < m; i++)
        {
            auto [x, y] = p[i];
            auto [u, v] = p[(i + 1) % m];
            long long a = v - y, b = x - u;
            lines.push_back({a, b, a * x + b * y});
        }
    }
    auto p = H::bounded_polygon(move(lines));
    long double area2 = 0;
    for (int i = 0; i < (int)p.size(); i++)
    {
        auto u = p[i], v = p[(i + 1) % p.size()];
        H::I numerator = H::I(u.x) * v.y - H::I(u.y) * v.x;
        H::I denominator = H::I(u.d) * v.d;
        area2 += (long double)numerator / (long double)denominator;
    }
    cout << fixed << setprecision(3) << fabsl(area2) / 2 << '\n';
}
```

第九章　平面最近点对

9.1　整数最近点对（精确平方距离）

代码依赖：IntegerPlane（第 14 页，1.8 节）。抄写时一并准备；标准库类型不列入算法依赖。

closest_pair_i64(points, endpoints=nullptr) 返回 optional<__int128_t> 的最短距离平方；传 &ids 时同时返回一对原输入的 0-based 下标，最优解不唯一时任选。少于两点返回 nullopt 且 ids={-1,-1}，重复点返回 0 和两个不同下标。坐标绝对值不超过 10^{12}。递归分治内部排序下标，不改变输入；返回时子区间按纵坐标有序。O(n log n) 时间、O(n) 辅助空间，全程精确整数。distance 更新全局端点，solve 返回局部最短平方距离；依赖 IntegerPlane。

```
optional<IntegerPlane::I> closest_pair_i64(const vector<IntegerPlane::Point> &p,
                                           pair<int, int> *endpoints = nullptr)
{
    using G = IntegerPlane;
    using I = G::I;
    int n = p.size();
    if (endpoints) *endpoints = {-1, -1};
    if (n < 2) return nullopt;
    vector<int> id(n), tmp(n);
    iota(id.begin(), id.end(), 0);
    sort(id.begin(), id.end(), [&](int a, int b) { return p[a] < p[b]; });
    for (int i = 1; i < n; i++)
        if (p[id[i]] == p[id[i - 1]])
        {
            if (endpoints) *endpoints = {id[i - 1], id[i]};
            return I(0);
        }
    I ans = I(1) << 120;
    auto distance = [&](int a, int b)
    {
        I d = G::dist2(p[a], p[b]);
        if (d < ans)
        {
            ans = d;
            if (endpoints) *endpoints = {a, b};
        }
        return d;
    };
    auto by_y = [&](int a, int b)
    {
        return tie(p[a].y, p[a].x) < tie(p[b].y, p[b].x);
    };
```

递归分治（接上页同一函数）：

```
    function<I(int, int)> solve = [&](int l, int r) -> I
    {
        if (r - l <= 3)
        {
            I best = I(1) << 120;
            for (int i = l; i < r; i++)
                for (int j = i + 1; j < r; j++)
                    best = min(best, distance(id[i], id[j]));
            sort(id.begin() + l, id.begin() + r, by_y);
            return best;
        }
        int m = (l + r) / 2;
        long long x = p[id[m]].x;
        I best = min(solve(l, m), solve(m, r));
        merge(id.begin() + l,
              id.begin() + m,
              id.begin() + m,
              id.begin() + r,
              tmp.begin() + l,
              by_y);
        copy(tmp.begin() + l, tmp.begin() + r, id.begin() + l);
        vector<int> strip;
        for (int i = l; i < r; i++)
            if ((I(p[id[i]].x) - x) * (I(p[id[i]].x) - x) < best)
            {
                for (int j = (int)strip.size() - 1; j >= 0; j--)
                {
                    I dy = I(p[id[i]].y) - p[strip[j]].y;
                    if (dy * dy >= best) break;
                    best = min(best, distance(id[i], strip[j]));
                }
                strip.push_back(id[i]);
            }
        return best;
    };
    return solve(0, n);
}
```

模板题使用：Luogu P7883

题意：求整数点集最近距离的平方，全程不取平方根。题目保证至少两点，坐标绝对值不超过 10^7，平方距离不超过 8×10^14，可将 int128 转为 long long；模板完整范围内不能无条件照搬转换。

所需模板：IntegerPlane（第 14 页，1.8 节）、closest_pair_i64（第 98 页，9.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P7883

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<IntegerPlane::Point> points(n);
    for (auto &p : points) cin >> p.x >> p.y;
    auto answer = closest_pair_i64(points);
    cout << (long long)*answer << '\n';
}
```

模板题使用：Library Checker closest_pair

题意：多组整数点集，输出一对不同的原输入 0-based 下标，使欧氏距离最小；最优解任选，重复坐标仍须返回不同下标。每组至少 2 点，总点数至 50 万，坐标绝对值至 10^9。直接传入端点指针，无需开方或重建坐标到编号的映射。

所需模板：IntegerPlane（第 14 页，1.8 节）、closest_pair_i64（第 98 页，9.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/closest_pair

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int tests;
    cin >> tests;
    while (tests--)
    {
        int n;
        cin >> n;
        vector<IntegerPlane::Point> p(n);
        for (auto &v : p) cin >> v.x >> v.y;
        pair<int, int> ids;
        closest_pair_i64(p, &ids);
        cout << ids.first << ' ' << ids.second << '\n';
    }
}
```

9.2 浮点平面最近点对

solve(points) 返回 long double 最短距离（不是平方），少于两个点返回 nullopt，重复点返回零。Point={x,y}；输入须有限，坐标绝对值 ≤1e100。递归分治，按纵坐标归并，O(n log n) 时间、O(n) 额外空间。用 hypotl 求距离，不先平方；排序不使用 epsilon。浮点运算仍有舍入误差，输入前已丢失的精度无法恢复。

```
struct ClosestPair
{
    using R = long double;

    struct Point
    {
        R x, y;
    };

    // Finite coordinates, |x|, |y| <= 1e100. Returns distance, not its square.
    static optional<R> solve(vector<Point> p)
    {
        int n = p.size();
        if (n < 2) return nullopt;
        sort(p.begin(),
             p.end(),
             [](Point a, Point b) { return tie(a.x, a.y) < tie(b.x, b.y); });
        for (int i = 1; i < n; i++)
            if (p[i].x == p[i - 1].x && p[i].y == p[i - 1].y) return R(0);
        auto by_y = [](Point a, Point b)
        {
            return tie(a.y, a.x) < tie(b.y, b.x);
        };
        auto distance = [](Point a, Point b)
        {
            return hypotl(a.x - b.x, a.y - b.y);
        };
        vector<Point> tmp(n);
        function<R(int, int)> dfs = [&](int l, int r) -> R
        {
            if (r - l <= 3)
            {
                R best = numeric_limits<R>::infinity();
                for (int i = l; i < r; i++)
                    for (int j = l; j < i; j++) best = min(best, distance(p[i], p[j]));
                sort(p.begin() + l, p.begin() + r, by_y);
                return best;
            }
```

分治、归并与跨分界线点对（接上页同一函数）：

```
            int m = (l + r) / 2;
            R x = p[m].x;
            R best = min(dfs(l, m), dfs(m, r));
            merge(p.begin() + l,
                  p.begin() + m,
                  p.begin() + m,
                  p.begin() + r,
                  tmp.begin() + l,
                  by_y);
            copy(tmp.begin() + l, tmp.begin() + r, p.begin() + l);
            vector<Point> strip;
            for (int i = l; i < r; i++)
                if (fabsl(p[i].x - x) <= best)
                {
                    for (int j = (int)strip.size() - 1; j >= 0; j--)
                    {
                        if (p[i].y - strip[j].y > best) break;
                        best = min(best, distance(p[i], strip[j]));
                    }
                    strip.push_back(p[i]);
                }
            return best;
        };
        return dfs(0, n);
    }
};
```

模板题使用：Luogu P1429

题意：求平面最近两点的距离，输出小数点后四位。solve 返回距离而非平方；题目保证至少两点，因此可解引用 optional。坐标按 long double 读取。

所需模板：ClosestPair（第 101 页，9.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P1429

```
int main()
{
    int n;
    scanf("%d", &n);
    vector<ClosestPair::Point> p(n);
    for (auto &v : p) scanf("%Lf%Lf", &v.x, &v.y);
    printf("%.4Lf\n", *ClosestPair::solve(p));
}
```

第十章　随机增量法

10.1　随机增量最小覆盖圆

代码依赖：RealPlane（第 1 页，1.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

solve(points,rng,eps) 返回 optional<Circle>，Circle={o,r}；空集返回 nullopt，单点或重复同一点返回半径零。rng 是调用者提供的 mt19937_64 随机数引擎。期望 O(n) 时间，最坏 $O(n^3)$，O(n) 空间；随机化不可省略。eps 默认 32 倍 long double 机器精度，用于相对半径的包含判断。diameter(a,b) 求直径圆；through(a,b,c) 求外接圆，共线时返回最远点对的直径圆。依赖浮点几何的 Point/Circle 与向量运算。输入及中间运算须有限且尺度适当；接近共线时圆心可能不稳定，无法恢复读入前的精度。

```
struct EnclosingCircle
{
    using G = RealPlane;
    using R = G::R;
    using Point = G::Point;
    using Circle = G::Circle;

    static Circle diameter(Point a, Point b)
    {
        Point o = a + (b - a) / 2;
        return {o, max(G::norm(o - a), G::norm(o - b))};
    }

    static Circle through(Point a, Point b, Point c)
    {
        Point u = b - a, v = c - a;
        R scale = max({fabsl(u.x), fabsl(u.y), fabsl(v.x), fabsl(v.y)});
        if (scale == 0) return {a, 0};
        u = u / scale;
        v = v / scale;
        R det = 2 * G::cross(u, v);
        if (det == 0)
        {
            Circle best = diameter(a, b);
            for (Circle q : {diameter(a, c), diameter(b, c)})
                if (q.r > best.r) best = q;
            return best;
        }
        R x = G::dot(u, u), y = G::dot(v, v);
        Point offset{(x * v.y - y * u.y) / det, (u.x * y - v.x * x) / det};
        Point o = a + offset * scale;
        return {o, max({G::norm(o - a), G::norm(o - b), G::norm(o - c)})};
    }
```

随机增量主过程（接上页同一结构体）：

```
    static optional<Circle>
    solve(vector<Point> p, mt19937_64 &rng, R eps = 32 * numeric_limits<R>::epsilon())
    {
        assert(eps >= 0 && eps < 1);
        if (p.empty()) return nullopt;
        shuffle(p.begin(), p.end(), rng);
        Circle c{p[0], 0};
        auto outside = [&](Point q)
        {
            return G::norm(q - c.o) > c.r * (1 + eps);
        };
        for (int i = 1; i < (int)p.size(); i++)
            if (outside(p[i]))
            {
                c = {p[i], 0};
                for (int j = 0; j < i; j++)
                    if (outside(p[j]))
                    {
                        c = diameter(p[i], p[j]);
                        for (int k = 0; k < j; k++)
                            if (outside(p[k])) c = through(p[i], p[j], p[k]);
                    }
            }
        return c;
    }
};
```

模板题使用：Luogu P1742

题意：求包含全部点的最小圆，先输出半径，再输出圆心坐标。题目保证非空，因此可解引用返回值；显式传随机引擎，不能省略随机打乱。用 long double 读取并输出十二位小数。

所需模板：RealPlane（第 1 页，1.1 节）、EnclosingCircle（第 103 页，10.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P1742

```
int main()
{
    int n;
    scanf("%d", &n);
    vector<EnclosingCircle::Point> p(n);
    for (auto &v : p) scanf("%Lf%Lf", &v.x, &v.y);
    mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
    auto c = *EnclosingCircle::solve(p, rng);
    printf("%.12Lf\n%.12Lf %.12Lf\n", c.r, c.o.x, c.o.y);
}
```

第十一章　配套知识与验证范围

11.1　数值算法与几何

排序必须使用严格弱序；不能以“差小于 eps 则相等”定义通用排序比较器。叉积与行列式容易发生抵消；整数题优先精确整数判定，浮点构造单独处理误差。自适应积分的局部误差估计需要函数足够规则；奇点、间断与强振荡必须先分段处理。三分搜索要求单峰性，整数域最后枚举剩余区间；二分答案要求判定单调性。

来源索引　本章为重新组织的公式速查，扩展范围依据 https://oi-wiki.org/math/。各算法实现、模板题用法与验证进度以仓库逐项清单为准；本章公式不能代替对应的完整代码与验证。

第十二章　计算几何：精确判定与适用范围

12.1　定点凸包、直径和极角排序

P2742 坐标最多两位小数，可按字符串读成放大 100 倍的整数，调用整数凸包；不要先读浮点再乘 100 截断。放大后坐标绝对值至多 10^8，满足整数几何范围。求周长时才对精确距离平方开方、最后除以 100；共线时取两端距离的两倍，全部重合为零。判定精确不等于最后的开方和小数输出没有舍入误差。

旋转卡壳输入须为严格逆时针凸包，不重复首点、不含共线边内点。用整数叉积比较三角形面积，并只在面积严格增大时推进对踵点；空集、单点和线段先单独处理。返回值是 128 位距离平方，不必开方再平方。P1452 的坐标绝对值至多 10^4，答案至多 8×10^8，其用法中可转 long long 输出；不能把这一转换推广到模板允许的万亿坐标。端点参数返回凸包下标，映射回原始编号见正式最远点对使用示例。

整数极角比较器从正 x 轴按 $[0,2\pi)$ 排序，同向按距离递增，零向量排最前。先分半圆，再比较精确叉积，最后比较长度；坐标轴归属必须明确。仅用 $y>0$ 分组会使 $(-1,0)<(0,-1)<(1,0)$ 却不满足首尾比较，破坏传递性。把 epsilon 相等用于排序也不满足严格弱序；不要把近似相等当作 sort 的等价关系。Library Checker 要求 $(-\pi,\pi]$，先按库排序，再把 $y<0$ 后缀循环移到最前；负 x 轴是 π，仍留在最后。原点约定角度为零。

凸包与卡壳用任意精度礼品包装、全点对距离作独立参考，含全部 3×3 子集、万亿坐标近共线和每个循环起点；极角比较器枚举 81 点的全部有序三元组，并与百位 atan2 逐对比较。三个应用驱动及极角正式题用法均双模式验证；极角排序全部 21 组官方本地数据双模式 checker 接受，不代表在线 AC。任意浮点输入的稳健凸包及极角谓词仍需单独补齐，不能由这些整数测试推断完成。

12.2 整数系数的有界半平面交

实现见第 94 页。输入是不等式 $ax + by \le c$，三个系数均为绝对值不超过 10^9 的整数。本接口要求交集有界，只输出正面积多边形；无解、点交集和线段交集都返回空向量。它不是一般的可行性分类接口，不可将其空返回直接解读为闭半平面交集必为空。无界输入不在约定范围内，也不偷偷添加一个人工边框。

两条非平行边界交点用齐次坐标 (X, Y, D) 表示：

$$D = a_1b_2 - b_1a_2, \quad X = c_1b_2 - b_1c_2, \quad Y = a_1c_2 - c_1a_2.$$

若 $D < 0$，三者同时取负，使 $D > 0$。测试 $aX + bY \le cD$ 即可判断交点位于半平面内，无需除法。系数界保证 $|X|, |Y|, D \le 2 \times 10^{18}$，可用 64 位保存；相等判断的交叉乘积至多为 4×10^{36}，使用 128 位。因此本接口内的构造、排序和位置判定都是精确整数运算。

按法向量的半圆与叉积排序，比较器不使用 epsilon。同方向平行约束按正规化后的常数项保留更紧者，避免除以法向量长度。扫描时若新约束排除队首或队尾交点，就删除相应无效边界；每条边界最多入队、出队一次，排序后为线性时间。最后闭合队列，清除重复顶点；有界的非正面积结果不会输出多边形。零法向量表示常真约束或直接矛盾，先单独处理。

测试枚举所有边界对的交点，用任意精度整数逐条验证可行性，将可行顶点集合与返回集合比较，并检查循环顺序的严格凸性。当前 vector 版本通过 5000 组有界随机输入、平行和退化案例、近乎平行的十亿系数，以及十万条冗余约束的规模测试；普通与 ASan/UBSan 均通过。P4196 的逆时针边 $(x, y) \to (u, v)$ 转换为 $a = v - y, b = x - u, c = ax + by$，则边的左侧就是 $ax' + by' \le c$。题目整数坐标绝对值至多 1000，故 $|a|, |b| \le 2000$、$|c| \le 2 \times 10^6$，满足模板范围。交集必有界；返回为空时，无论空集或零面积退化，所求面积都是零。鞋带公式每项分子 $X_iY_j - Y_iX_j$、分母 D_iD_j 先以 128 位求值，再转浮点相除累加。最多 500 个交点且坐标绝对值不超过 1000，满足本题输出精度。完整驱动另经 207 组精确有理数逐边裁剪对拍，包含近共线、点/线段交和 500 条边界。这不保证一般大坐标有理多边形面积的统一相对误差，浮点输入也未由此验证。WIDA 的一般半平面交条目保留部分完成状态，浮点版本与无界分类仍需补齐。

12.3 最近点对、Minkowski 和与三维整数判定

整数最近点对见第 98 页；Minkowski 和见第 83 页。最近点对返回距离平方，不开方；少于两个点时返回空 optional，重复点返回零。分治使用递归，每层按纵坐标归并，总时间 $O(n \log n)$，额外空间 $O(n)$。坐标绝对值上限为 10^{12}，作差后平方使用 128 位。

Minkowski 和输入必须是严格逆时针凸包，不重复首点，不含边上的冗余共线点；允许循环移动起点，也允许空集、单点和线段。输入坐标以及任意两个输入点的坐标和都须在 $[-10^{12}, 10^{12}]$ 内。非退化凸包按边方向归并，时间 $O(n+m)$；退化分支枚举点和后求凸包，时间 $O((n+m)\log(n+m))$。返回同样不重复首点的严格凸包。

两套代码通过任意精度整数距离枚举，以及对所有点和独立执行礼品包装法的对拍；覆盖退化、循环起点和大坐标。规模测试包含二十万点最近点对，以及 40001 顶点的抛物线凸包与自身求和，结果应为原凸包各坐标的两倍。这些属于本地验证，不表示新增在线 AC。

三维判定见第 44 页。点的三个坐标为 `x,y,z`，绝对值均不超过 10^9。`orient(a,b,c,d)` 返回有向六倍四面体体积，零表示四点共面，交换任意两个点会改变符号。`collinear` 判断三点共线，`on_segment` 包含端点，且支持零长度线段。传统版线段接口名为 `On_Segment`。

点差每个分量至多 2×10^9，叉积分量至多 8×10^{18}，三重积绝对值至多 4.8×10^{28}，在有符号 128 位范围内。公开的叉积、点积辅助函数不对任意 128 位向量保证不溢出；上述保证针对点差及其叉积。任意精度整数按 4×4 行列式的 24 个排列独立计算方向，并用二阶子式及坐标区间验证共线与在线段上。已通过立方体八个顶点的全部有序四元组、随机十亿级坐标、换序及退化测试，普通与 ASan/UBSan 均通过。这里不包含三维凸包构造。

12.4 浮点最近点对

实现见第 101 页。`ClosestPair::solve` 与传统版 `Closest_Pair::Solve` 接收 `Point{x,y}` 数组，返回最短距离，不是距离平方。少于两点返回空 optional；重复点返回零。输入为有限 `long double`，坐标绝对值不超过 10^{100}。

递归前按横坐标排序，每次在递归改变数组前保存分界横坐标。左右区间分别递归后按纵坐标归并，再检查分界线附近的点对。时间 $O(n \log n)$，额外空间 $O(n)$。直接使用 `hypotl`，避免先平方造成上溢或下溢。排序比较器不使用 epsilon；不能把相近坐标随意视为相同点。

浮点结果仍有舍入误差，也无法恢复读入之前已经丢失的精度。整数坐标要求精确判定时，优先使用整数最近点对，最后按题目要求开方输出。一百位十进制浮点枚举所有点对，验证两套实现，覆盖小数、相邻可表示浮点数、极小非正规数、10^{-100} 至 10^{100} 的尺度，以及二十万点规模；普通与 ASan/UBSan 均通过。本地完整 P1429 驱动测试包含 103 组输入和四位小数输出检查。在线评测的整数输入范围不能替代这里对小数与极端尺度的检查。

两套实现已在 P1429 通过全部 11 个测试点：动态版记录 297602650、传统版记录 297603238。单点最大用时均为 75 ms，内存峰值均为 12.32 MB。

12.5 公切线与过点切线

实现见第 25 页，依赖浮点几何的点、圆和向量运算。令两圆心差为 v，$d = |v|$，$u = v/d$。外公切线取 $s = 1$，内公切线取 $s = -1$，单位法向量满足

$$n \cdot u = \frac{r_1 - s r_2}{d} = c, \qquad n = cu \pm \sqrt{(1-c)(1+c)}\, u^{\perp}.$$

切点为 $p = o_1 + r_1 n$、$q = o_2 + s r_2 n$。直线以 $n \cdot (x - p) = 0$ 表示，避免在两圆相切、两个切点相同时丢失方向。结果中的 `side` 为 1 或 -1，表示外公切线或内公切线。每个非相切分支有两条线，相切分支只输出一次。

半径零解释为点。有一个点圆时，只枚举外公切线分支，避免内外分支重复；两个不同点直接返回连线，`side=0`。严格同心且等半径返回 `infinite=true`，包括重合的两个点圆；严格同心但半径不同没有公切线。`from_point(p,c)` 把第一圆设为点圆，圆上的切点读返回的 `b`。点在圆上时返回一条，点在圆内时为空；点与零半径圆重合时返回无限条过点直线。

输入须有限，坐标和半径绝对值不超过 10^{100}，半径非负。容差 $0 < \varepsilon < 1$ 默认 10^{-12}：当 $|r_1 - s r_2|$ 与 d 的相对差在容差内时，合并为相切。因此极接近相切的输入可能返回一条近似切线，不承诺恢复舍入前的几何分类；使用方程时宜计算 $n \cdot (x - p)$，避免先计算大常数项造成额外消减。先比较半径差和圆心距，再做除法，避免极小圆心距造成比值溢出。

计数对拍使用任意精度整数比较圆心距平方与半径和、差的平方；一百位十进制运算验证切点到圆心的距离、圆心到切线的距离、法向量和切线一致性。两套实现通过 3000 组随机圆，在三个尺度下交换顺序测试，并覆盖重合、内外相切、点圆、极近同心与容差边界。普通与 ASan/UBSan 均通过；尚无此接口的在线 AC 记录。

12.6 随机增量最小覆盖圆

实现见第 103 页。随机打乱点集后依次加入。新点在当前圆外时，它必须成为新覆盖圆的边界点；以该点为固定边界重扫前缀，再遇到圆外点就固定两个边界点，最后由第三个圆外点确定外接圆。随机化使期望时间为 $O(n)$，最坏时间仍为 $O(n^3)$。传入初始化好的 `mt19937_64`，不可省略打乱。本实现使用三层循环，没有递归栈需求。

空集返回空 optional，单点及重复同一点的半径为零。直径圆用两端点到计算圆心的较大距离作为半径。三点外接圆先把第一点平移到原点，再按最大坐标差缩放后解两条垂直平分线方程；共线时使用最远点对的直径圆。`through` 对非共线三点求外接圆，不是任意三点的最小覆盖圆；例如钝角三角形的最小覆盖圆可能仅由最长边支撑，须调用主过程。

包含判断使用相对半径容差，默认 32 倍机器精度。输入与中间运算须有限、尺度适当；近共线三点的外接圆仍可能数值敏感，浮点比较不能恢复读入之前的精度。P1742 的驱动已准备，输出半径与圆心，该题坐标绝对值不超过 10^4，最多两位小数。

本地参考答案枚举所有单点、点对直径圆和三点外接圆，用一百位十进制运算筛出覆盖全部点且半径最小者，检查圆心、半径及覆盖性。两套版本通过 700 组随机小数点集，每组使用 12 个随机种子，另测共线、近共线、钝角/锐角三角形、平移后的小三角形、十万点直线与重复点。普通与 ASan/UBSan 均通过。两套代码已在 P1742 通过全部 10 个测试点：动态版记录 297608403、传统版记录 297608801。单点最大用时分别为 52 ms、49 ms，内存峰值为 6.50 MB、6.49 MB。该评测验证半径与圆心，不覆盖一并展开但未调用的其他几何接口。随机增量算法可参考 Project Nayuki 的算法说明。

12.7 圆与简单多边形相交面积

实现见第 23 页。先把圆心移到原点，再沿多边形有向边累加贡献，最后取绝对值。边与圆的交点把边分成至多三段。若小段中点在圆内，贡献是 $\operatorname{cross}(p,q)/2$；否则贡献是有向扇形面积

$$\frac{r^2}{2}\operatorname{atan2}\big(\operatorname{cross}(p,q),p\cdot q\big).$$

必须保留符号，不能先对每条边取绝对值，否则凹多边形和圆外的边无法抵消。使用补偿求和减小大量边累加时的误差，总时间 $O(n)$、额外空间 $O(1)$。

求交点时令边的单位方向为 u，有向距离 $h=\operatorname{cross}(u,a)$，垂足直接构造为 $hu^{\perp}$，交点为

$$hu^{\perp}\pm u\sqrt{(r-|h|)(r+|h|)}.$$

这避免用远端点加很长的位移来重建圆附近的交点。沿边的投影参数用于筛选位于线段内部的交点；端点本身已在分段列表内。相切交点重复不改变面积，零长度边返回零。

多边形须简单，支持凹形、两种方向和相邻重复顶点；半径非负，零半径或少于三个顶点时面积为零。输入与中间运算须有限且尺度适当，特别是平移后的乘积和面积要可表示。极小相交面积可能受到扇形贡献相消和近相切计算的影响，不作相对误差的统一保证。

测试包括完整圆、半圆、四分之一圆、内含多边形、相离、相切和高精度圆缺面积。独立参考实现沿竖直方向求多边形截线区间，再与圆的截线相交并数值积分，检查 150 组含凹形的随机多边形，另检查方向反转、重复点和平移。规模与数值测试含十万条边、极长的过圆心边和不同尺度。两套实现普通与 ASan/UBSan 均通过；尚无此模块的在线 AC。对应 kuangbin 二维几何中的圆与三角形、多边形相交面积部分。

12.8 最近点对编号与多边形闭集查询

最近点对可返回原输入的两个不同下标，重复点距离为零。不足两点返回空值，端点重置为 $(-1,-1)$。内部排序下标，不改变输入序列。分治递归前保存中间横坐标，递归后各段按纵坐标排列，再归并和扫描窄带。局部距离用于剪枝，全局更新保存端点；同距最优解任选。坐标至 10^{12} 时距离平方可达 8×10^{24}，需用 int128；P7883 的坐标至 10^7 才允许在用法中转 long long。Library Checker closest_pair 的原编号输出见完整用法。

一般简单多边形直接保留输入边界调用 polygon_contains，不把凹形先变成凸包。返回 0/1/2 分别表示外部、边界、内部。先判在线段上，再用半开纵坐标区间累计绕数；边界反向不改变结果。凸包二分查询要求严格逆时针、无共线边内点、不重复首点，允许旋转起点；扇形内部对角线不等于多边形边界。

P4557 中两组点的凸包记为 A,B，第二组平移 d 后相交，当且仅当 $d\in A+(-B)$。先对第二组坐标取负，再求两凸包的 Minkowski 和，对原平移向量做闭集查询。不要把查询也取负；边界接触输出 1。各组点数及查询数至 10^5，结果坐标至 2×10^8；复杂度为 $O(n\log n+m\log m+q\log(n+m))$。这是比赛应用，不替代两个组件的正式非比赛模板题。

独立检查使用整数斜射线奇偶、凸包直接相交、全点对距离枚举；最近点对另通过 29 组官方数据的双模式本地 checker。本地接受记录与线上 AC、速度排名分开保留。

12.9 整数圆交点：精确判定与浮点构造

输入整数坐标绝对值、半径均不超过 10^9，半径可以为零。直线方向 $v=b-a$，令 $q=v\cdot v$、$c=v\times(a-o)$，判别式为 $D=r^2q-c^2$。$q=0$ 是退化直线；否则按 D 的符号精确分类。垂足 $h=o+\mathrm{perp}(v)c/q$，交点为 $h\pm v\sqrt{D}/q$。两圆则令 $v=b-a$、$q=v\cdot v$、$k=q+r^2-s^2$，用 $D=4r^2q-k^2$ 分类，交点为 $a+vk/(2q)\pm\mathrm{perp}(v)\sqrt{D}/(2q)$。$q=0$ 单独处理同心圆；相同正半径有无穷交点，相同零半径仅有一点。

这些整数中间值均在有符号 int128 范围内：$q\le 8\times10^{18}$，$|c|\le 8\times10^{18}$，$|k|\le 9\times10^{18}$；$c^2\le 6.4\times10^{37}$、$k^2\le 8.1\times10^{37}$，$4r^2q\le 3.2\times10^{37}$。差的两项非负，差值也安全。必须先提升到 int128 再做减法、乘法；扩大输入范围需重新核算。

精确分类不意味着坐标无舍入误差。AOJ CGL_7_D/E 的输入尺度仅为 10^4，本地按严格绝对 10^{-6} 检查；不能将此误差承诺扩展到 10^9。相切时核心只返回一个点，题目用法复制一次；按精确字典序排序浮点结果，不能用 eps 比较器。原浮点容差版仍用于明确接受近似分类的场景：例如圆心 $(-7717,-10000)$、半径 9758，直线经过 $(-10000,-367),(-5909,-108)$，整数判别式为 2168，两交点相距约 0.0227，旧容差却会归为相切，因此不能直接用于该题。

CGL_7_H 的多边形允许凹形，直接调用圆与多边形面积；CGL_7_I 包含同心、内含与相离圆，直接调用两圆交面积。本轮完整用法用 80 位精度参考核对，H 采用独立竖切片解析积分，并检查完整圆、半圆、四分之一圆锚点。局部验证、线上 AC 与速度排名分别记录。

12.10 公切线：切点次序也需要精确判定

CircleTangents 的 line.a、line.b 分别是第一、第二圆的切点。from_point 把给定点视为第一圆，因此 CGL_7_F 输出 line.b。该题整数范围为 10^3，点严格在圆外；非零的 d^2-r^2 至少为 1，故 $d-r\ge 1/(d+r)>2\times10^{-4}$，远大于默认相对容差所对应的距离。

CGL_7_G 则需按第一圆切点的横坐标、纵坐标输出。圆 $(0,0,3)$ 与 $(5,5,4)$ 有两个切点 $(1.8,-2.4),(1.8,2.4)$，浮点运算可能分别产生 1.7999999999999998 与 1.7999999999999985，直接排序会错排。IntegerTangents 精确决定输出次序，构造坐标后不再排序。

设圆心差为 $v=(x,y)$，$q=x^2+y^2$，外、内公切线分别取 $g=r-s$ 与 $g=r+s$，$D=q-g^2$；$D<0$ 无此类线，$D=0$ 一条，否则两条。单位法向为 $n=(vg\pm\mathrm{perp}(v)\sqrt{D})/q$。第一圆半径为正时，比较切点等价于比较法向的横、纵分量。每次比较化为 $A+B\sqrt{C}+E\sqrt{F}$ 的符号：先判断 $u=A+B\sqrt{C}$ 和 $w=E\sqrt{F}$，同号可直接决定；异号时比较 $u^2-w^2=(A^2+B^2C-E^2F)+2AB\sqrt{C}$，再乘 $\mathrm{sgn}(u)$。单根式也先判符号，再平方比较大小，避免丢失符号和伪等号。

输入坐标绝对值、半径至 10^4 时，$|A|\le 4\times10^8$，$|B|,|E|\le 2\times10^4$，$C,F\le 8\times10^8$。第二次比较的两项平方分别不超过 2.304×10^{35} 与 2.048×10^{35}，有符号 int128 可容纳；不能直接沿用圆交点函数的十亿范围。这是严格的根式比较器，没有对浮点横坐标作 eps 比较或取整分组。零半径、同心、相切与空结果沿用几何意义；法向仍可定义相切时的唯一切线，不要用两个重合切点计算方向。输入超范围或非整数时，应另行选择数值合同。

12.11 圆交点：容差分类与坐标误差

默认浮点接口按相对容差处理近切情况，不能从“距离误差小”推出“交点坐标误差小”。接近相切时，半弦长包含平方根，归并成一个点可能放大坐标误差。直线–圆与圆–圆函数现可显式传 `eps=0` 使用无容差分支，默认正容差分支不变；这不等于任意实数上的精确几何。实现及完整用法分别见第 8、10、9、12 页。

避免先开方再相减。 直线令 $v = b - a$、$q = v \cdot v$、$c = v \times (a - o)$，计算 $\Delta = r^2 q - c^2$。若 $q = 0$ 则直线退化；否则 Δ 的负、零、正分别对应无交点、相切和两个交点。垂足 $h = o + \mathrm{perp}(v)c/q$，偏移 $w = v\sqrt{\Delta}/q$，返回 $h \pm w$。两圆令 $v = b.o - a.o$、$q = v \cdot v$、$k = q + r^2 - s^2$，计算 $\Delta = 4r^2 q - k^2$；同心情形先单独处理。此时 $h = a.o + vk/(2q)$、$w = \mathrm{perp}(v)\sqrt{\Delta}/(2q)$。这些公式没有复用整数接口，也没有把默认 eps 简单调小。

AOJ 整数输入域的额外保证。 CGL_7_D/E 给出的坐标为绝对值不超过 $B = 10^4$ 的整数，半径为 $1..B$ 的整数。$q \le 8B^2$、$|c| \le 8B^2$、$|k| \le 9B^2$，上述每一步整数运算的绝对值均小于 $81B^4 < 2^{60}$。若 long double 为二进制且有效尾数至少 64 位，采用正常最近舍入，不启用 fast-math、危险重结合或改变精度的环境，这些中间整数和判别式符号均精确。53 位尾数不够：$9805^2 \cdot 234901757 - 150276382^2 = 1$，但直接 53 位浮点计算可得到零。整数十亿坐标接口的范围不能挪给本结论；一般非整数输入也不继承它。

构造与排序。 设 $u = 2^{-64}$。有交点时 h – 圆心 与 w 的每个坐标绝对值均不超过 B。在算术正确舍入、`sqrtl` 误差至多一 ulp 的条件下，按除法、乘法、加法逐步估计，交点每坐标误差小于 $20uB < 1.1 \times 10^{-14}$；固定 12 位小数再增加至多 5×10^{-13}，仍远小于题目 10^{-6}。这是一项带前提的误差分析，不把本地样本最大误差冒充全域证明。不同交点的非零横坐标间隔，直线情形至少 $2/q$，两圆情形至少 $1/q \ge 1.25 \times 10^{-9}$，故上述误差不会颠倒顺序。相等横坐标来自方向的相应整数分量为零，计算结果也相等。直接按 (x, y) 字典序排序，不使用 epsilon 比较器；相切时按题面重复输出同一点。

范围之外。 `eps=0` 仅取消主动归并。其他实数和更大尺度上，判别式仍可能消去、上溢或下溢，交点构造仍可能病态。默认接口的近切反例继续保留，不宣称已获得通用稳健浮点圆交点。`RealGeometry` 兼容构造仍要求正 eps，因为它同时服务其他几何函数。

12.12 三维投影、旋转与平面求交的数值约定

三维向量见第 61 页，直线与线段见第 64 页，平面见第 67 页。四份接口用法分别是第 63、66、69、70 页；目前没有该完整接口的正式模板题 AC。分类参考OI Wiki 三维计算几何基础。

`Line3(p,d)` 的第二参数是方向，表示 $p+td$；两个端点用 `Line3::through(a,b)` 构造。投影到无限直线时参数不截断，投影到线段时限制 $0 \le t \le 1$。零方向按点 p 处理距离和投影，不能作旋转轴。向量排序使用精确字典序，禁止以 epsilon 作为排序的相等条件。

对单位轴方向 u 和 $v = q - p$，右手旋转为

$$p + v\cos\theta + (u \times v)\sin\theta + u(u \cdot v)(1 - \cos\theta).$$

绕正 z 轴旋转 $\pi/2$ 时，正 x 轴转到正 y 轴；角度单位是弧度。`angle(a,b)` 返回非零向量的 $[0,\pi]$ 夹角；无向平面夹角另取 $\min(\theta, \pi - \theta)$。

平面保存点 p 与单位法向 n，方程为 $n\cdot(x-p)=0$。`equation(n,h)` 表示 $n\cdot x = h$，一般式 $Ax+By+Cz+D=0$ 应传 $h=-D$。法向必须非零，三点构造必须不共线。有符号距离是 $n\cdot(q-p)$，投影为 $q-n[n\cdot(q-p)]$。负法向翻转距离符号，不改变投影。

求交返回 1 表示单点或交线，0 表示无交，-1 表示整条线在面上或两平面重合；仅返回 1 时写输出参数。零方向线在面上时返回单点，不能返回整条线。两单位法向 n, m 非平行时，取 $d = \mathrm{unit}(n \times m)$、$v = d \times n$，两平面公共点可以取

$$q = p + v\frac{m \cdot (f.p - p)}{m \cdot v}, \qquad \text{交线为} q + td.$$

分母不能随意取绝对值，交线点不要求最靠近原点。

默认距离容差为 10^{-10}，角度正弦容差为 10^{-12}。线面按 $|n \cdot u|$，面面按 $|n \times m|$ 判近乎平行，再用代表点距离分类。这是一套数值规则，近乎平行的对象在精确意义上可能仍然相交；改变坐标尺度、代表点或容差可能改变阈值附近的分类。浮点求交不提供精确拓扑证书。需要精确共面谓词时使用第 44 页的整数版本并遵守其坐标界。

12.12.1 原三维片段中需要避开的错误

kuangbin 原 188 行逐页核对后，保留完整夹具以记录以下缺陷：

- 平面角写成 $\arccos(n \cdot m)/(|n||m|)$，括号位置错误；法向 $(2,0,0)$ 与 $(0,2,0)$ 得到 $\pi/8$ 而不是 $\pi/2$。
- 交线分母取绝对值后丢失方向：$x = 1$ 与 $y = 2$ 得到的点不在第二平面上。
- 闭线段的点积条件写成等于零，漏掉中点；零线段距离还出现 0/0。
- epsilon 字典序不满足传递性，不能用于 `sort`；输出函数误用了 `scanf`。

新实现不复刻原稿固定 epsilon 混用不同量纲的行为。坐标必须有限，所有中间量须避免溢出和下溢；`hypot` 只改善长度计算，不能替整个算法保证任意尺度稳定。巨大平移与微小差值、近共线构造和近平行求交都可能放大误差，应先平移/缩放并按题意选容差。本机 GNU16 的 `long double` 尾数只有 53 位，不能假定它总比 `double` 更精确。

测试包含 4999 组非退化直线来源对照、8014 组完整接口数据，以有理数投影、四元数旋转和两平面关联条件作独立参考。普通、关闭断言、展开与抄写形式均运行，ASan/UBSan 通过；漏截断、旋转方向、法向归一化、交线位移符号四种错误变体被检出。这些是数值验证证据，不是对所有实数输入正确分类或线上资源通过的保证。

算法索引

circle_coverage_area, 40
circle_intersections, 10
circle_intersections_i64, 18
circle_overlap_area, 13
circle_union_area, 36
CirclePolygon, 23
CircleTangents, 25
closest_pair_i64, 98
ClosestPair, 101
convex_contains_i64, 82
convex_diameter2, 91
convex_tangents_i64, 75

EnclosingCircle, 103
Euler characteristic, 73

Hull3D, 46
Hull3DMetrics, 56

integer_hull, 79
IntegerGeometry3D, 44
IntegerHalfplanes, 94
IntegerPlane, 14
IntegerTangents, 27

lattice polygon, 73
Line3, 64
line_circle_i64, 16
line_circle_intersections, 8
line_intersection_real, 6
line_projection, 5

minkowski_sum, 83

numerical stability, 105

Pick theorem, 73
Plane3, 67
polygon_area2, 19
polygon_contains, 22
polygon_union_area, 30

RealPlane, 1
RealPolarLess, 3
RealSpace, 61
rectangle_union_area, 89

segment_distance_real, 71
SupportHull, 85